

Build a Full-Stack Project Manager

Flaxon + Teloce HTML SPA
Signals, Admin/CMS, PWA, and MinifyJS

Aldane Hutchinson

Learner ebook and recording companion | Revision 6 | October 2026

Build protected APIs first. Connect the interface to working data. Verify the whole application before deployment.

Start with the CLI. Follow exact file edits, run each chapter, and build a working application through deployment.

Contents

01 | Preview and CLI setup

02 | Settings, management and application composition

03 | Models and Python migrations

04 | Customer authentication and sessions

05 | Owned project APIs

06 | Task workflow

07 | Backend tests

08 | Teloce HTML shell and scoped CSS

09 | Login and project screens

10 | Task components, signals and progress

11 | SPA routing and direct refresh

12 | Staff Admin

13 | CMS help and sample data

14 | Full-stack verification

15 | Production build and Render

16 | PWA: installation, offline fallback and updates

Appendix | Recording this application

Read this first

Start with `flaxon new project_manager`. Build in that generated directory; keep the separate course_reference checkout for pinned dependencies and recovery files. Use Python 3.12 and basic Python, HTML and JavaScript knowledge. You do not need Node.js for this course.

Follow chapters in order. Stop the development server before replacing Python files. Follow the action label on each step. CREATE supplies the complete new file. REPLACE THE ENTIRE FILE supplies a complete replacement. EDIT MARKED BLOCKS shows an exact existing block and its replacement: find the old block once and replace only that block, keeping everything else. Never append a duplicate function or route. Complete recovery files are available for every chapter. Create parent folders when they do not exist. Empty __init__.py blocks mean create an empty file. Commands run inside project_manager unless a chapter explicitly says otherwise. Once a check passes, commit your work before the next lesson.

The recording target is Flaxon 3.0.0. Its release installation commands become usable only after publication and a clean release rehearsal. Until then, the supplied preview wheels are the verified installation path; they are not labelled as a published 3.0.0 package. Keep the supplied requirements and vendor wheels together. Their checksums are verified before installation. This revision uses Python ORM migrations and management.py throughout. Older chapter tags describe the previous course revision and remain unchanged.

Each lesson gives a goal, an explanation, exact files, commands, expected results, common errors and a short exercise. For recording, demonstrate the expected result, explain the boundary being changed, type the important behavior and run its check. The final recording appendix is optional; you can learn the application directly from the chapters.

Finished application

Chapter 16 adds optional PWA installation, a connection-status banner, an offline fallback and explicit updates. It never queues private task changes offline.

Customer pages: /login, /projects, /projects/ID, /help and /help/SLUG. Customers register, manage their own projects and tasks, filter status and see progress. Staff pages: /admin/login, /admin/project, /admin/task and /admin/cms/. Staff permission checks run on the server. Each Teloce component has scoped CSS. Internal SPA anchors use data-teloce-link; staff navigation remains ordinary page navigation.

01 | Preview and CLI setup

Goal

Generate your own project and run its welcome screen.

What you are building

You will build a project manager with customer accounts, private projects, tasks, progress, a staff Admin and published help. Flaxon runs Python on the server. Teloce compiles HTML components and TypeScript into a browser SPA. MinifyJS optimizes the production JavaScript. We build the APIs first so every screen connects to a working backend.

Choose the installation before recording

The public course targets **Flaxon 3.0.0**. The release is planned, not published by this book. After publication and the clean release rehearsal, learners can install it from PyPI instead of copying preview wheels. Do not mix the two installation paths.

Release path: after Flaxon 3.0.0 is published

Use a new parent folder. Create and activate an environment as shown below, then install the release:

```
python -m venv course_env
# macOS/Linux: source course_env/bin/activate
# Windows PowerShell: .\course_env\Scripts\Activate.ps1
python -m pip install "flaxon[standard,admin]==3.0.0"
flaxon new project_manager
cd project_manager
# Activate the generated .venv using your platform's command:
# macOS/Linux: source .venv/bin/activate
# Windows PowerShell: .\.venv\Scripts\Activate.ps1
python -m pip install "flaxon[standard,admin]==3.0.0" minifyjs==0.1.3 pytest==9.1.1
    httpx==0.28.1 playwright==1.51.0
python -m pip check
python -m flaxon welcome
python -m flaxon run app:app --reload
```

CREATE `requirements.txt`**:** after the release passes the course checks, copy the complete contents of `requirements-release.txt` from the course repository into the generated project. Keep that file in Git. Chapters 2-16 then use the same application code and commands. Skip `prepare_course.py` and the preview wheel-copying commands on this path. Chapter 15 deployment uses your release requirements; its vendor-verification helper runs only for the preview path.

Say: "We installed the published course version, then generated our own project. Flaxon already includes the ORM integration. We still apply migrations explicitly."

Viewers should see: the generated welcome screen and a successful Python API request. If the exact release cannot be installed, it has not been published on the configured package index; use the labelled preview path to rehearse, not a different version presented as 3.0.0.

Preview path: verified bundled build before release

The following existing bootstrap installs the bundled preview. It is for rehearsal until the release path passes the same checks. It does not install Flaxon 3.0.0.

Start here: create the project with the CLI

Use Python 3.12. Run the first commands in a new parent directory, not inside the finished course repository. The reference checkout supplies the pinned course wheels; you will write application code yourself in the generated project.

```
git clone https://github.com/aldanedev-create/FreecodeCamp-flaxon-project-manager-course.git
    course_reference
python -m venv course_env
```

Activate the bootstrap environment on Windows PowerShell:

```
.\course_env\Scripts\Activate.ps1
```

Or activate it on macOS/Linux:

```
source course_env/bin/activate
```

Install while inside the reference folder so the relative wheel paths resolve:

```
cd course_reference
python -m pip install -r requirements-dev.txt
cd ..
flaxon new project_manager
cd project_manager
```

The CLI creates the project and its own `.venv`. Activate that project environment now (rather than keeping the parent environment for the rest of the course).

Windows PowerShell:

```
.\.venv\Scripts\Activate.ps1
```

macOS/Linux:

```
source .venv/bin/activate
```

CREATE: prepare_course.py

Create this file in the generated project. It copies dependency files only; it does not copy the completed application's Python or UI code.

```
from pathlib import Path
import shutil

reference = Path("../course_reference")
if not reference.is_dir():
    raise SystemExit("Keep course_reference beside project_manager.")

shutil.copytree(reference / "vendor", "vendor", dirs_exist_ok=True)
for name in ("requirements.txt", "requirements-dev.txt", "requirements.lock.txt"):
    shutil.copy2(reference / name, name)

Path("scripts").mkdir(exist_ok=True)
shutil.copy2(reference / "scripts/verify_vendor.py", "scripts/verify_vendor.py")
print("Pinned course dependencies are ready; application code is still the starter.")
```

Run these commands now

```
python prepare_course.py
python scripts/verify_vendor.py
python -m pip install -r requirements-dev.txt
python -m flaxon welcome
python -m flaxon welcome-status
python -m flaxon run app:app --reload
```

Open http://127.0.0.1:8000/. Show the welcome screen and click its Python API button. Do not set up staff or run the generated migration yet. Chapter 3 supplies the ORM models and generates its first Python migration.

What to say and show at this point

Say: "We generated the starter ourselves. The dependency helper copied wheels and version pins, not the finished task manager. Next we will build the backend one module at a time. The customer SPA returns in chapter 8."

Show: the welcome module, `flaxon_cli.py`, `management.py`, and `ui/app.html`. Explain that the generated starter has its own CSS file; chapter 8 replaces the shell and removes that file in favour of scoped component CSS.

Stop the server with Ctrl+C before chapter 2. Initialize your own checkpoints with `git init` and `git add .`, then `git commit -m "Chapter 01 setup"`. Do not create a nested project_manager directory by running the generation command again inside it.

Expected result

The welcome page loads. Click its API button and see the Python response. The welcome command prints the project name.

Common errors

If flaxon is missing, activate the environment or use python -m flaxon. Environment creation does not install application dependencies; run the separate installation commands. Keep course_reference beside project_manager. Do not generate inside the completed repository.

Short exercise

Find the welcome module's route and the HTML component that calls it.

What to say and show

Say: "Generate your own project and run its welcome screen. The server remains responsible for persistence and authorization; the browser presents the result."

Show the expected result above, then point to the file responsible for it. Run the chapter check before the next lesson.

Save your checkpoint

```
git add .
git commit -m "Complete chapter 01: Preview and CLI setup"
```

02 | Settings, management and application composition

Goal

Replace the welcome shell with a small backend and keep one shared configuration.

What you are building

settings.py reads environment values and gives both the server and management commands the same database location. app.py creates an application from those settings and mounts modules explicitly. management.py delegates to Flaxon; you do not maintain a second command framework. The factory returns early in management mode so migration commands do not compile browser assets or open the staff store. For now, the only mounted feature is the generated welcome module. Later chapters add each module when its code is ready.

Build this chapter

Stop the server. Work in project_manager. Follow each CREATE, whole-file replacement, or marked-block edit below in order.

Step 1: EDIT - replace the entire file - settings.py

Read this first: choose paths and environment values in one place. The rest of the app uses these values.

Select all existing contents of this file and replace them with the complete code below.

```
"""Shared configuration for the server and management commands."""

from pathlib import Path
from urllib.parse import urlsplit
from flaxon.config import env

BASE_DIR = Path(__file__).resolve().parent
ROOT = BASE_DIR
env.load(BASE_DIR / ".env")
PROJECT_NAME = "Project Manager"
DATA_DIR = Path(env.str("DATA_DIR", str(BASE_DIR / "data"))).resolve()
DATABASE_PATH = DATA_DIR / "app.sqlite3"
ADMIN_DATABASE_PATH = DATA_DIR / "admin.sqlite3"
DATABASE_URL = env.str("DATABASE_URL", f"sqlite://{DATABASE_PATH}")
DEBUG = env.bool("FLAXON_DEBUG", default=True)
SECRET_KEY = env.str("FLAXON_SECRET_KEY")
PUBLIC_ORIGIN = env.str("PUBLIC_ORIGIN", "http://127.0.0.1:8000").rstrip("/")
ALLOWED_HOSTS = env.list(
    "FLAXON_ALLOWED_HOSTS", default=[urlsplit(PUBLIC_ORIGIN).hostname, "testserver"]
)
CSRF_TRUSTED_ORIGINS = [PUBLIC_ORIGIN]
TIME_ZONE = "UTC"
ADMIN_ENABLED = True
CMS_ENABLED = True
ADMIN_SERVICES_ENABLED = False
ADMIN_STORE_BACKEND = "sqlite"
ADMIN_STORAGE_PATH = ADMIN_DATABASE_PATH
```

Say: "Read this first: choose paths and environment values in one place. The rest of the app uses these values."

Type: Follow the action above in `settings.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 2: EDIT - replace the entire file - management.py

This short entry point delegates commands to Flaxon and fixes the project root.

Select all existing contents of this file and replace them with the complete code below.

```
"""One entry point for Flaxon's project commands."""

from pathlib import Path
from flaxon.management import execute


def main(argv=None):
    return execute(argv, project_root=Path(__file__).resolve().parent)


if __name__ == "__main__":
    raise SystemExit(main())
```

Say: "This short entry point delegates commands to Flaxon and fixes the project root."

Type: Follow the action above in `management.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 3: EDIT - replace the entire file - app.py

The application factory mounts only the features already introduced. Follow the action label: a marked-block edit keeps the surrounding factory intact.

Select all existing contents of this file and replace them with the complete code below.

```
"""Compose the CLI starter's modules, staff backoffice, and Teloce SPA."""

import settings
from types import SimpleNamespace
from pathlib import Path
from urllib.parse import urlsplit
from flaxon.middleware import BodyLimitMiddleware, TrustedHostsMiddleware
from flaxon import Flaxon, Request
from flaxon.http import JSONResponse
from modules.welcome.module import welcome
from settings import (
    ROOT,
    DATA_DIR,
    DATABASE_PATH,
    ADMIN_DATABASE_PATH,
    DEBUG,
    PUBLIC_ORIGIN,
)


def create_app(database_path=None, admin_path=None, debug=None):
    debug = DEBUG if debug is None else debug
```

```
    values = {key: getattr(settings, key) for key in dir(settings) if key.isupper()}
    values.update(DEBUG=debug)
    if database_path is not None:
        values["DATABASE_URL"] = f"sqlite://{Path(database_path).resolve()}"
    if admin_path is not None:
        values["ADMIN_STORAGE_PATH"] = Path(admin_path)
    app = Flaxon.from_settings(SimpleNamespace(__file__=settings.__file__, **values))
    app.add_middleware(BodyLimitMiddleware, max_size=256 * 1024)
    if not debug:
        app.add_middleware(
            TrustedHostsMiddleware, allowed_hosts=[urlsplit(PUBLIC_ORIGIN).hostname]
        )

    app.public_origin = PUBLIC_ORIGIN
    for module, prefix in [
        (welcome, "/api/welcome"),
    ]:
        app.mount_module(module, prefix=prefix)
    if app.is_management:
        return app


    @app.get("/health/course")
    async def health():
        return {"data": {"status": "ok"}}

    @app.get("/")
    async def home():
        return {"data": {"message": "Backend lesson ready. The SPA starts in chapter 8."}}

    return app


app = create_app()
```

Say: "The application factory mounts only the features already introduced. Follow the action label: a marked-block edit keeps the surrounding factory intact."

Type: Follow the action above in `app.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Run and check

```
python management.py runserver
# In a second terminal:
curl http://127.0.0.1:8000/api/welcome/status
```

Expected result

The root returns a Backend lesson ready JSON message. /api/welcome/status returns its timestamp and framework version.

Common errors

Run commands inside project_manager. Do not place module mounts in settings.py. Keep the generated models.py until chapter 3; the ORM discovers it but does not create tables on startup.

Short exercise

Change PROJECT_NAME in settings.py and inspect app.settings in a Python session.

What to say and show

Say: "Replace the welcome shell with a small backend and keep one shared configuration. The server remains responsible for persistence and authorization; the browser presents the result."

Show the expected result above, then point to the file responsible for it. Run the chapter check before the next lesson.

Save your checkpoint

```
git add .
git commit -m "Complete chapter 02: Settings, management and application composition"
```

03 | Models and Python migrations

Goal

Define all five application tables and apply the first Python migration.

What you are building

User owns projects; Project owns tasks. The ForeignKeyField names use the root ORM label models. Cascading a project deletion removes its tasks. Session stores a digest of an opaque cookie and an expiry; AuthAttempt persists login attempt counts. Define the schema once in models.py. makemigrations compares models with migration history and generates Python code; migrate applies that code. Commit the generated file. No JSON migration is needed. We define the complete schema now so later API chapters focus on behavior rather than repeated table changes.

Build this chapter

Stop the server. Work in project_manager. Follow each CREATE, whole-file replacement, or marked-block edit below in order.

Step 1: CREATE - make parent folders, then create this file - models.py

These five models are the complete schema. Relationships define ownership and deletion behavior; handlers add authorization.

Create the parent directories, then save this complete file.

```
"""Application tables. Generate Python migrations whenever these models change."""

from flaxon.db import Model, fields


class User(Model):
    id = fields.IntField(primary_key=True)
    name = fields.CharField(max_length=80)
    email = fields.CharField(max_length=254, unique=True)
    password_hash = fields.TextField()

    class Meta:
        table = "users"

    def __str__(self):
        return self.name
```

```
class Project(Model):
    id = fields.IntField(primary_key=True)
    owner = fields.ForeignKeyField(
        "models.User", related_name="projects", on_delete=fields.CASCADE
    )
    name = fields.CharField(max_length=120)
    description = fields.TextField(default="")
    created_at = fields.DatetimeField(auto_now_add=True)

    class Meta:
        table = "projects"

    def __str__(self):
        return self.name

class Task(Model):
    id = fields.IntField(primary_key=True)
    project = fields.ForeignKeyField(
        "models.Project", related_name="tasks", on_delete=fields.CASCADE
    )
    title = fields.CharField(max_length=200)
    status = fields.CharField(max_length=10, default="todo")
    due_date = fields.DateField(null=True)
    created_at = fields.DatetimeField(auto_now_add=True)

    class Meta:
        table = "tasks"

    def __str__(self):
        return self.title


class Session(Model):
    token_hash = fields.CharField(max_length=64, primary_key=True)
    user = fields.ForeignKeyField("models.User", null=True, on_delete=fields.CASCADE)
    csrf = fields.CharField(max_length=64)
    expires_at = fields.BigIntField(db_index=True)

    class Meta:
        table = "sessions"


class AuthAttempt(Model):
    key = fields.CharField(max_length=64, primary_key=True)
    attempts = fields.IntField(default=0)
    started_at = fields.BigIntField(db_index=True)

    class Meta:
        table = "auth_attempts"
```

Say: "These five models are the complete schema. Relationships define ownership and deletion behavior; handlers add authorization."

Type: Follow the action above in `models.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 2: EDIT - replace the entire file - admin.py

Keep explicit registration in this file. At chapter 3 it is empty; chapter 12 adds the domain models.

Select all existing contents of this file and replace them with the complete code below.

```
"""Staff model registration is added in chapter 12."""
def register(admin):
    pass
```

Say: "Keep explicit registration in this file. At chapter 3 it is empty; chapter 12 adds the domain models."

Type: Follow the action above in `admin.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 3: CREATE - make parent folders, then create this file - migrations/__init__.py

This file owns the feature named by its module or component. Read the route/form flow before continuing.

Create the parent directories, then save this complete file.

Say: "This file owns the feature named by its module or component. Read the route/form flow before continuing."

Type: Follow the action above in `migrations/__init__.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 4: EDIT - replace the marked blocks - app.py

The application factory mounts only the features already introduced. Follow the action label: a marked-block edit keeps the surrounding factory intact.

Edit 1: find this exact block

Open `app.py`. Locate this block once. The unchanged surrounding lines identify its position; do not use line numbers from another revision.

```
    @app.get("/health/course")
    async def health():
        return {"data": {"status": "ok"}}

    @app.get("/")
```

Replace that block with

```
    @app.get("/health/course")
    async def health():
        from models import User

        await User.all().limit(1)
        return {"data": {"status": "ok"}}

    @app.get("/")
```

Leave the rest of the file unchanged.

Say: "The application factory mounts only the features already introduced. Follow the action label: a marked-block edit keeps the surrounding factory intact."

Type: Follow the action above in `app.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Run and check

```
python management.py check
python management.py makemigrations --name initial
python management.py migrate --plan
python management.py migrate
python management.py migrate --status
python management.py runserver
```

Flaxon API and ORM: the commands you will teach

This course uses the current Flaxon public integration rather than JSON migrations or hand-written SQL. Flaxon's `Model` and `fields` come from Tortoise; queries are asynchronous. The Flaxon 3.0.0 release target must export the same APIs before recording the release installation path.

Say: "Models describe tables. Settings choose the database. Flaxon opens and closes connections. Migrations record schema changes, and management.py applies them explicitly."

In `models.py`, the exact imports are:

```
from flaxon.db import Model, fields
```

You have already created `Project` in this chapter. After migrating, use these queries in an async handler or the management shell. These examples explain the API; do not paste them at module scope, and do not add an unsecured demonstration route:

```
project = await Project.create(
    owner_id=user["id"], name="Record the course", description="First project"
)
projects = await Project.filter(owner_id=user["id"]).order_by("-created_at")
project = await Project.filter(id=project_id, owner_id=user["id"]).first()
await Project.filter(id=project_id, owner_id=user["id"]).update(name="New name")
await Project.filter(id=project_id, owner_id=user["id"]).delete()
```

`user` is the authenticated customer dictionary and `project_id` is the validated route parameter introduced in chapters 4-5. Never trust an owner ID supplied by the browser. A model relationship does not perform authorization automatically.

For an operation that must commit or roll back together:

```
from flaxon.db import in_transaction

async with in_transaction() as connection:
    project = await Project.create(
        owner_id=user["id"], name="Atomic project", using_db=connection
    )
    await Task.create(
        project_id=project.id, title="First task", using_db=connection
    )
```

The server manages normal connection startup and shutdown through `Flaxon.from_settings(...)`. The course tests create disposable schemas explicitly; normal application startup never silently

changes tables.

Keep these responsibilities separate:

- `settings.py`: `DATABASE_URL`, debug mode, secrets and deployment configuration.
- `models.py`: fields, foreign keys, indexes and readable model names.
- `app.py`: `Flaxon.from_settings(...)`, middleware and `app.mount_module(...)`.
- `management.py`: one short entry point backed by `flaxon.management.execute`.
- `admin.py`: explicit staff model registration; chapter 12 adds controlled adapters.

Run these from `project_manager`:

```
python management.py check
python management.py makemigrations --name initial
python management.py migrate --plan
python management.py migrate
python management.py migrate --status
python management.py shell
python management.py setup-admin
python management.py runserver
```

Only run `makemigrations --name initial` once for this initial schema. Later changes use a descriptive new name. `createsuperuser` is an alias for `setup-admin`; migration files are Python and belong in Git. Stop the server before changing configuration. Staff credentials and customer accounts are separate.

The later chapters show actual `FlaxonModule` definitions, JSON responses, request parsing, middleware, Teloce compilation and Admin/CMS integration in their complete files. Optional SSR is outside this course's client-rendered SPA; use the dedicated framework guide when adding it.

Expected result

check reports valid settings and registrations. makemigrations creates migrations/0001_initial.py. migrate applies it; status lists it as applied. A second migrate performs no new schema changes.

Common errors

Use a fresh learner project. Do not apply the starter ProjectNote migration before replacing models.py. If you already migrated the starter, select a NEW DATA_DIR in .env and start the course migration history in a separate project. Do not delete or reset a database containing data you need. No such table means the selected database has not been migrated.

Short exercise

Add a priority field to Task in a disposable copy. Generate and inspect a second migration without applying it to your course database.

What to say and show

Say: "Define all five application tables and apply the first Python migration. The server remains responsible for persistence and authorization; the browser presents the result."

Show the expected result above, then point to the file responsible for it. Run the chapter check before the next lesson.

Save your checkpoint

```
git add .
git commit -m "Complete chapter 03: Models and Python migrations"
```

04 | Customer authentication and sessions

Goal

Register, sign in, sign out and reject requests without valid sessions.

What you are building

validation.py checks JSON and field types before handlers write anything. security.py reads hashed session tokens, compares CSRF tokens, checks browser origins and rotates sessions after authentication changes. Password hashing runs in a worker thread because Argon2 is CPU intensive. The login handler verifies a dummy hash for unknown emails. The durable limiter counts attempts inside a transaction. Customer accounts are separate from staff Admin accounts. Start by requesting /api/auth/session; this issues an anonymous cookie and CSRF token. Registration and login replace both, so use the new values for the next request.

Build this chapter

Stop the server. Work in project_manager. Follow each CREATE, whole-file replacement, or marked-block edit below in order.

Step 1: CREATE - make parent folders, then create this file - validation.py

Validate JSON before passing values to ORM calls. Reuse these checks in all mutations.

Create the parent directories, then save this complete file.

```
"""Validate every write on the server, even when the browser already validated it."""

import re
from datetime import date
from flaxon.exceptions import BadRequest


async def json_object(request):
    try:
        data = await request.json()
    except (ValueError, UnicodeDecodeError):
        raise BadRequest("Send a valid JSON object.")
    if not isinstance(data, dict):
        raise BadRequest("Send a JSON object.")
    return data


def text(data, key, maximum=120, required=True):
    value = data.get(key, "")
    if not isinstance(value, str):
        raise BadRequest(f"{key} must be text.")
```

```
    value = value.strip()
    if (required and not value) or len(value) > maximum:
        raise BadRequest(
            f"{key} must contain 1 to {maximum} characters."
            if required
            else f"{key} must contain at most {maximum} characters."
        )
    return value


def credentials(data):
    email = text(data, "email", 254).lower()
    if not re.fullmatch(r"[^\s@]+@[^\s@]+\.[^\s@]+", email):

        raise BadRequest("Enter a valid email address.")
    password = data.get("password")
    if not isinstance(password, str) or not 12 <= len(password) <= 128:
        raise BadRequest("Use a password with 12 to 128 characters.")
    return email, password


def task_fields(data, partial=False):
    result = {}
    if not partial or "title" in data:
        result["title"] = text(data, "title", 160)
    if not partial or "status" in data:
        status = data.get("status", "todo")
        if status not in ("todo", "doing", "done"):
            raise BadRequest("Status must be todo, doing, or done.")
        result["status"] = status
    if not partial or "due_date" in data:
        due = data.get("due_date") or None
        if due is not None:
            try:
                due = date.fromisoformat(due).isoformat()
            except (ValueError, TypeError):
                raise BadRequest("Use YYYY-MM-DD for a due date.")
        result["due_date"] = due
    if not result:
        raise BadRequest("Supply at least one editable field.")
    return result
```

Say: "Validate JSON before passing values to ORM calls. Reuse these checks in all mutations."

Type: Follow the action above in `validation.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 2: CREATE - make parent folders, then create this file - security.py

Read each helper in order: identify a session, require its user, check CSRF, rotate credentials, then limit attempts.

Create the parent directories, then save this complete file.

```
"""Opaque cookie sessions and session-bound CSRF; account ownership stays on the server."""

import asyncio
import hashlib
import hmac
import secrets
import time
```

```
from models import User, Session, AuthAttempt
from tortoise.expressions import F
from flaxon.db import in_transaction
from flaxon.exceptions import Forbidden, Unauthorized, TooManyRequests
from flaxon.http import JSONResponse
from flaxon.http.cookies import Cookie

COOKIE_NAME = "project_session"
SESSION_SECONDS = 8 * 60 * 60


def digest(value):
    return hashlib.sha256(value.encode()).hexdigest()


async def read_session(request):
    token = request.cookies.get(COOKIE_NAME, "")
    if not token:
        return None
    return (
        await Session.filter(token_hash=digest(token), expires_at__gt=int(time.time()))
        .first()
        .values()
    )


async def require_user(request):

    session = await read_session(request)
    if not session or session["user_id"] is None:
        raise Unauthorized("Please sign in.")
    user = (
        await User.filter(id=session["user_id"]).first().values("id", "name", "email")
    )
    if user is None:
        raise Unauthorized("Please sign in.")
    return user


async def check_csrf(request):
    origin = request.headers.get("origin")
    if origin and origin != request.app.public_origin:
        raise Forbidden("This request came from an untrusted origin.")
    if "application/json" not in request.headers.get("content-type", "").lower():
        raise Forbidden("Send JSON with a CSRF token.")
    session = await read_session(request)
    supplied = request.headers.get("x-csrf-token", "")
    if (
        not session
        or not supplied
        or not hmac.compare_digest(session["csrf"], supplied)
    ):
        raise Forbidden("Your session expired. Reload the page and try again.")


async def session_response(request, user=None, status=200):
    old = request.cookies.get(COOKIE_NAME, "")
    token, csrf = secrets.token_urlsafe(32), secrets.token_urlsafe(32)
    now = int(time.time())
    async with in_transaction():
        if old:
            await Session.filter(token_hash=digest(old)).delete()

        await Session.filter(expires_at__lte=now).delete()
```

```
        await Session.create(
            token_hash=digest(token),
            user_id=user["id"] if user else None,
            csrf=csrf,
            expires_at=now + SESSION_SECONDS,
        )
    cookie = Cookie(
        COOKIE_NAME,
        token,
        path="/",
        httponly=True,
        samesite="Lax",
        secure=not request.app.debug,
        max_age=SESSION_SECONDS,
    )
    response = JSONResponse(
        {"data": {"user": user, "csrf": csrf}},
        status_code=status,
        headers={"cache-control": "no-store"},
    )
    response.headers.add("set-cookie", cookie.to_header())
    return response


async def limit_auth_attempts(request, email):
    """A durable, single-instance limiter; production proxies also limit the auth routes."""
    peer = request.scope.get("client") or ("unknown", 0)
    key = digest(str(peer[0]) + ":" + email)
    now = int(time.time())

    async with in_transaction():
        await AuthAttempt.filter(started_at__lt=now - 900).delete()
        await AuthAttempt.get_or_create(key=key, defaults={"started_at": now})

        await AuthAttempt.filter(key=key).update(attempts=F("attempts") + 1)
        record = await AuthAttempt.get(key=key)
        attempts = record.attempts
    if attempts > 10:
        raise TooManyRequests("Too many attempts. Try again in 15 minutes.")
```

Say: "Read each helper in order: identify a session, require its user, check CSRF, rotate credentials, then limit attempts."

Type: Follow the action above in `security.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 3: CREATE - make parent folders, then create this file - modules/auth/__init__.py

This file owns the feature named by its module or component. Read the route/form flow before continuing.

Create the parent directories, then save this complete file.

```
"""Module-owned APIs and interface files."""
```

Say: "This file owns the feature named by its module or component. Read the route/form flow before continuing."

Type: Follow the action above in `modules/auth/__init__.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 4: CREATE - make parent folders, then create this file - modules/auth/module.py

Create the auth module with session, registration, login and logout routes. Start with the anonymous CSRF handshake; registration rotates the cookie and token.

Create the parent directories, then save this complete file.

```
"""Registration and cookie authentication, separate from staff Admin accounts."""

import asyncio
from tortoise.exceptions import IntegrityError
from models import User
from pathlib import Path
from argon2.exceptions import VerifyMismatchError, VerificationError
from flaxon.modules import FlaxonModule
from flaxon.exceptions import Conflict, Unauthorized
from flaxon.http import JSONResponse
from security import (
    check_csrf,
    read_session,
    require_user,
    session_response,
    limit_auth_attempts,
)
from validation import json_object, text, credentials

auth = FlaxonModule(
    "auth", ui_dir=Path(__file__).parent / "ui", ui_routes={"Login.js": "/login"}
)


@auth.get("/session")
async def session(request):
    current = await read_session(request)
    if current:
        user = await require_user(request) if current["user_id"] else None
        return JSONResponse(
            {"data": {"user": user, "csrf": current["csrf"]}},
            headers={"cache-control": "no-store"},
        )
    return await session_response(request)

@auth.post("/register")
async def register(request):
    await check_csrf(request)
    data = await json_object(request)
    email, password = credentials(data)
    name = text(data, "name", 80)
    await limit_auth_attempts(request, email)
    password_hash = await asyncio.to_thread(request.app.hasher.hash, password)
    try:
        user = await User.create(name=name, email=email, password_hash=password_hash)
        user_id = user.id
    except IntegrityError:
        raise Conflict("Unable to create this account. Try signing in.")
    return await session_response(
```

```
        request, {"id": user_id, "name": name, "email": email}, status=201
    )


@auth.post("/login")
async def login(request):
    await check_csrf(request)
    email, password = credentials(await json_object(request))
    await limit_auth_attempts(request, email)
    user = await User.filter(email=email).first().values()
    password_hash = user["password_hash"] if user else request.app.dummy_password_hash
    try:
        await asyncio.to_thread(request.app.hasher.verify, password_hash, password)
    except (VerifyMismatchError, VerificationError):
        raise Unauthorized("Email or password is incorrect.")
    if user is None:
        raise Unauthorized("Email or password is incorrect.")
    public_user = {key: user[key] for key in ("id", "name", "email")}

    return await session_response(request, public_user)


@auth.post("/logout")
async def logout(request):
    await check_csrf(request)
    return await session_response(request)
```

Say: "Create the auth module with session, registration, login and logout routes. Start with the anonymous CSRF handshake; registration rotates the cookie and token."

Type: Follow the action above in `modules/auth/module.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 5: EDIT - replace the marked blocks - app.py

The application factory mounts only the features already introduced. Follow the action label: a marked-block edit keeps the surrounding factory intact.

Edit 1: find this exact block

Open `app.py`. Locate this block once. The unchanged surrounding lines identify its position; do not use line numbers from another revision.

```
from pathlib import Path
from urllib.parse import urlsplit
from flaxon.middleware import BodyLimitMiddleware, TrustedHostsMiddleware
from flaxon import Flaxon, Request
from flaxon.http import JSONResponse
from modules.welcome.module import welcome
from settings import (
    ROOT,
```

Replace that block with

```
from pathlib import Path
from urllib.parse import urlsplit
from flaxon.middleware import BodyLimitMiddleware, TrustedHostsMiddleware
from argon2 import PasswordHasher
from flaxon import Flaxon, Request
from flaxon.http import JSONResponse
from modules.auth.module import auth
from modules.welcome.module import welcome
from settings import (
    ROOT,
```

Leave the rest of the file unchanged.

Edit 2: find this exact block

Open `app.py`. Locate this block once. The unchanged surrounding lines identify its position; do not use line numbers from another revision.

```
            TrustedHostsMiddleware, allowed_hosts=[urlsplit(PUBLIC_ORIGIN).hostname]
        )
    app.public_origin = PUBLIC_ORIGIN
    for module, prefix in [
        (welcome, "/api/welcome"),
    ]:
        app.mount_module(module, prefix=prefix)
```

Replace that block with

```
            TrustedHostsMiddleware, allowed_hosts=[urlsplit(PUBLIC_ORIGIN).hostname]
        )
    app.public_origin = PUBLIC_ORIGIN
    app.hasher = PasswordHasher()
    app.dummy_password_hash = app.hasher.hash("dummy-password-for-timing-only")
    for module, prefix in [
        (auth, "/api/auth"),
        (welcome, "/api/welcome"),
    ]:
        app.mount_module(module, prefix=prefix)
```

Leave the rest of the file unchanged.

Say: "The application factory mounts only the features already introduced. Follow the action label: a marked-block edit keeps the surrounding factory intact."

Type: Follow the action above in `app.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Run and check

```
python management.py runserver
# In a second terminal:
curl -c cookies.txt http://127.0.0.1:8000/api/auth/session
```

Make the first authenticated request

Keep the server running. In a second terminal use an HTTP client to GET /api/auth/session and save its Set-Cookie value and data.csrf. Then POST /api/auth/register with that cookie, Content-Type: application/json and X-CSRF-Token set to data.csrf. The complete Python demo in chapter 5 automates this exchange. For curl on macOS/Linux:

```
curl -c cookies.txt http://127.0.0.1:8000/api/auth/session
# Replace TOKEN with data.csrf from the response above:
curl -b cookies.txt -c cookies.txt -H 'Content-Type: application/json' -H 'X-CSRF-Token:
    TOKEN' -d
    '{"name":"Learner","email":"learner@example.test","password":"LearningPython123!"}'
    http://127.0.0.1:8000/api/auth/register
```

Save the NEW token returned by registration. Do not commit cookies.txt.

Expected result

GET /api/auth/session returns data.user = null and a CSRF token. POST /api/auth/register with the cookie and token creates a customer. GET /api/projects is introduced next; the auth module itself now supports login and logout.

Common errors

Use PUBLIC_ORIGIN exactly, including scheme, hostname and port. A 403 can mean a missing or rotated CSRF token, wrong Origin or non-JSON body. Passwords must have 12-128 characters. Never put the HttpOnly cookie in localStorage.

Short exercise

Use curl or a small HTTP client to register, then log out and show that the previous cookie cannot authenticate.

What to say and show

Say: "Register, sign in, sign out and reject requests without valid sessions. The server remains responsible for persistence and authorization; the browser presents the result."

Show the expected result above, then point to the file responsible for it. Run the chapter check before the next lesson.

Save your checkpoint

```
git add .
git commit -m "Complete chapter 04: Customer authentication and sessions"
```

05 | Owned project APIs

Goal

Create, list, read, update and delete projects belonging to the signed-in customer.

What you are building

owned_project combines the requested ID with the authenticated owner's ID in the ORM query. Every detail, edit and delete uses that guard. Creation takes owner_id from the session rather than accepting it from JSON. list_projects filters before returning rows. A nonexistent project and another person's project both return 404. The small HTTP demo establishes cookies and CSRF automatically, registers an account and exercises a working project endpoint.

Build this chapter

Stop the server. Work in project_manager. Follow each CREATE, whole-file replacement, or marked-block edit below in order.

Step 1: CREATE - make parent folders, then create this file - modules/projects/__init__.py

This file owns the feature named by its module or component. Read the route/form flow before continuing.

Create the parent directories, then save this complete file.

```
"""Module-owned APIs and interface files."""
```

Say: "This file owns the feature named by its module or component. Read the route/form flow before continuing."

Type: Follow the action above in `modules/projects/__init__.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 2: CREATE - make parent folders, then create this file - modules/projects/module.py

Create the projects module, then implement owned_project before the handlers that use it. The mount adds /api/projects once. Every ORM query for an existing project includes the signed-in customer's owner ID.

Create the parent directories, then save this complete file.

```
"""Project APIs always constrain queries by the signed-in user's id."""

from pathlib import Path
from models import Project
from flaxon.modules import FlaxonModule
from flaxon.http import JSONResponse
from flaxon.exceptions import NotFound
from security import require_user, check_csrf
```

```
from validation import json_object, text

projects = FlaxonModule(
    "projects",
    ui_dir=Path(__file__).parent / "ui",
    ui_routes={
        "ProjectList.js": "/projects",
        "ProjectDetails/[id].js": "/projects/:id",
    },
)


async def owned_project(request, project_id):
    user = await require_user(request)
    project = await Project.filter(id=project_id, owner_id=user["id"]).first().values()
    if project is None:
        raise NotFound("Project not found.")
    return project


@projects.get("/")
async def list_projects(request):
    user = await require_user(request)
    items = await Project.filter(owner_id=user["id"]).order_by("-id").values()
    return {"data": items}

@projects.post("/")
async def create_project(request):
    await check_csrf(request)
    user = await require_user(request)
    data = await json_object(request)
    name = text(data, "name")
    description = text(data, "description", 1000, required=False)
    project = await Project.create(
        owner_id=user["id"], name=name, description=description
    )
    project_id = project.id
    return JSONResponse(
        {"data": await owned_project(request, project_id)}, status_code=201
    )


@projects.get("/<int:project_id>")
async def get_project(request, project_id):
    return {"data": await owned_project(request, project_id)}


@projects.put("/<int:project_id>")
async def update_project(request, project_id):
    await check_csrf(request)
    project = await owned_project(request, project_id)
    data = await json_object(request)
    name = text(data, "name")
    description = text(data, "description", 1000, required=False)
    await Project.filter(id=project_id, owner_id=project["owner_id"]).update(
        name=name, description=description
    )
    return {"data": await owned_project(request, project_id)}

@projects.delete("/<int:project_id>")
async def delete_project(request, project_id):
    await check_csrf(request)
    project = await owned_project(request, project_id)
    await Project.filter(id=project_id, owner_id=project["owner_id"]).delete()
```

```
    return {"data": {"deleted": True}}
```

Say: "Create the projects module, then implement owned_project before the handlers that use it. The mount adds /api/projects once. Every ORM query for an existing project includes the signed-in customer's owner ID."

Type: Follow the action above in `modules/projects/module.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 3: CREATE - make parent folders, then create this file - scripts/course_api_demo.py

This file owns the feature named by its module or component. Read the route/form flow before continuing.

Create the parent directories, then save this complete file.

```
"""Show the project API over HTTP; start the development server first."""
import secrets
import httpx


def main():
    # Each run creates its own learner and project in your development database.
    with httpx.Client(base_url="http://127.0.0.1:8000", trust_env=False) as client:
        response = client.get("/api/auth/session")
        response.raise_for_status()
        csrf = response.json()["data"]["csrf"]
        response = client.post(
            "/api/auth/register",
            headers={"X-CSRF-Token": csrf},
            json={"name": "Sample Learner", "email":
    f"sample-{secrets.token_hex(6)}@example.test", "password": "LearningPython123!"},
        )
        response.raise_for_status()
        csrf = response.json()["data"]["csrf"]
        response = client.post(
            "/api/projects/", headers={"X-CSRF-Token": csrf},
            json={"name": "My first project", "description": "Record the sample lesson"},
        )
        response.raise_for_status()
        assert response.status_code == 201
        project = response.json()["data"]
        print("Created:", project)
        response = client.get(f"/api/projects/{project['id']}")
        response.raise_for_status()
        print("Read:", response.json()["data"])
        response = client.post("/api/projects/", headers={"X-CSRF-Token": csrf}, json={"name":
     " "})
        assert response.status_code == 400
        print("Invalid input rejected:", response.status_code)

        print("Use your own browser account to create a separate project on /projects.")


if __name__ == "__main__":
    main()
```

Say: "This file owns the feature named by its module or component. Read the route/form flow before continuing."

Type: Follow the action above in `scripts/course_api_demo.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 4: EDIT - replace the marked blocks - app.py

The application factory mounts only the features already introduced. Follow the action label: a marked-block edit keeps the surrounding factory intact.

Edit 1: find this exact block

Open `app.py`. Locate this block once. The unchanged surrounding lines identify its position; do not use line numbers from another revision.

```
from flaxon import Flaxon, Request
from flaxon.http import JSONResponse
from modules.auth.module import auth
from modules.welcome.module import welcome
from settings import (
    ROOT,
```

Replace that block with

```
from flaxon import Flaxon, Request
from flaxon.http import JSONResponse
from modules.auth.module import auth
from modules.projects.module import projects
from modules.welcome.module import welcome
from settings import (
    ROOT,
```

Leave the rest of the file unchanged.

Edit 2: find this exact block

Open `app.py`. Locate this block once. The unchanged surrounding lines identify its position; do not use line numbers from another revision.

```
    app.dummy_password_hash = app.hasher.hash("dummy-password-for-timing-only")
    for module, prefix in [
        (auth, "/api/auth"),
        (welcome, "/api/welcome"),
    ]:
        app.mount_module(module, prefix=prefix)
```

Replace that block with

```
    app.dummy_password_hash = app.hasher.hash("dummy-password-for-timing-only")
    for module, prefix in [
        (auth, "/api/auth"),
        (projects, "/api/projects"),
        (welcome, "/api/welcome"),
    ]:
        app.mount_module(module, prefix=prefix)
```

Leave the rest of the file unchanged.

Say: "The application factory mounts only the features already introduced. Follow the action label: a marked-block edit keeps the surrounding factory intact."

Type: Follow the action above in `app.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Run and check

```
python management.py runserver
# In a second terminal:
python scripts/course_api_demo.py
```

Expected result

The HTTP demo creates a project and reads it back. Projects return a data envelope with the owner, name and description. A second customer's request cannot retrieve it.

Common errors

Do not repeat /api/projects inside decorators: app.py supplies that mount prefix. Use a trailing slash for the list/create endpoint. A 401 means no valid customer session; 404 after sign-in can be an ownership denial.

Short exercise

Extend the demo to update a project's description and assert the read response changed.

What to say and show

Say: "Create, list, read, update and delete projects belonging to the signed-in customer. The server remains responsible for persistence and authorization; the browser presents the result."

Show the expected result above, then point to the file responsible for it. Run the chapter check before the next lesson.

Save your checkpoint

```
git add .
git commit -m "Complete chapter 05: Owned project APIs"
```

06 | Task workflow

Goal

Add tasks, update status and due dates, filter tasks and delete them.

What you are building

Tasks belong to an owned project. list_tasks checks that project before querying its tasks. owned_task checks the parent project before permitting any mutation. A PATCH validates only supplied fields; allowed statuses are todo, doing and done. Date-only values use YYYY-MM-DD or null. The ORM handles values as parameters rather than SQL fragments. Deleting a project cascades through the database relationship.

Build this chapter

Stop the server. Work in project_manager. Follow each CREATE, whole-file replacement, or marked-block edit below in order.

Step 1: CREATE - make parent folders, then create this file - modules/tasks/__init__.py

This file owns the feature named by its module or component. Read the route/form flow before continuing.

Create the parent directories, then save this complete file.

```
"""Module-owned APIs and interface files."""
```

Say: "This file owns the feature named by its module or component. Read the route/form flow before continuing."

Type: Follow the action above in `modules/tasks/__init__.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 2: CREATE - make parent folders, then create this file - modules/tasks/module.py

Create the tasks module. Require ownership of the parent project before listing, creating, editing or deleting tasks. Validate status and due dates before saving.

Create the parent directories, then save this complete file.

```
"""Tasks belong to projects; project ownership guards every task operation."""

from models import Task
from flaxon.modules import FlaxonModule
from flaxon.http import JSONResponse
from flaxon.exceptions import NotFound, BadRequest
from security import check_csrf
from validation import json_object, task_fields
from modules.projects.module import owned_project
```

```
tasks = FlaxonModule("tasks")


async def owned_task(request, task_id):
    task = await Task.filter(id=task_id).first().values()
    if task is None:
        raise NotFound("Task not found.")
    await owned_project(request, task["project_id"])
    return task


@tasks.get("/project/<int:project_id>")
async def list_tasks(request, project_id):
    await owned_project(request, project_id)
    status = request.query.get("status")
    if status and status not in ("todo", "doing", "done"):
        raise BadRequest("Unknown task status.")
    query = Task.filter(project_id=project_id)
    if status:
        query = query.filter(status=status)
    return {"data": await query.order_by("-id").values()}


@tasks.post("/project/<int:project_id>")

async def create_task(request, project_id):
    await check_csrf(request)
    await owned_project(request, project_id)
    data = task_fields(await json_object(request))
    task = await Task.create(project_id=project_id, **data)
    task_id = task.id
    return JSONResponse({"data": await owned_task(request, task_id)}, status_code=201)


@tasks.patch("/<int:task_id>")
async def update_task(request, task_id):
    await check_csrf(request)
    current = await owned_task(request, task_id)
    data = task_fields(await json_object(request), partial=True)
    await Task.filter(id=task_id, project_id=current["project_id"]).update(**data)
    return {"data": await owned_task(request, task_id)}


@tasks.delete("/<int:task_id>")
async def delete_task(request, task_id):
    await check_csrf(request)
    await owned_task(request, task_id)
    await Task.filter(id=task_id).delete()
    return {"data": {"deleted": True}}
```

Say: "Create the tasks module. Require ownership of the parent project before listing, creating, editing or deleting tasks. Validate status and due dates before saving."

Type: Follow the action above in `modules/tasks/module.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 3: EDIT - replace the marked blocks - app.py

The application factory mounts only the features already introduced. Follow the action label: a marked-block edit keeps the surrounding factory intact.

Edit 1: find this exact block

Open app.py. Locate this block once. The unchanged surrounding lines identify its position; do not use line numbers from another revision.

```
from flaxon.http import JSONResponse
from modules.auth.module import auth
from modules.projects.module import projects
from modules.welcome.module import welcome
from settings import (
    ROOT,
```

Replace that block with

```
from flaxon.http import JSONResponse
from modules.auth.module import auth
from modules.projects.module import projects
from modules.tasks.module import tasks
from modules.welcome.module import welcome
from settings import (
    ROOT,
```

Leave the rest of the file unchanged.

Edit 2: find this exact block

Open app.py. Locate this block once. The unchanged surrounding lines identify its position; do not use line numbers from another revision.

```
    for module, prefix in [
        (auth, "/api/auth"),
        (projects, "/api/projects"),
        (welcome, "/api/welcome"),
    ]:
        app.mount_module(module, prefix=prefix)
```

Replace that block with

```
    for module, prefix in [
        (auth, "/api/auth"),
        (projects, "/api/projects"),
        (tasks, "/api/tasks"),
        (welcome, "/api/welcome"),
    ]:
        app.mount_module(module, prefix=prefix)
```

Leave the rest of the file unchanged.

Say: "The application factory mounts only the features already introduced. Follow the action label: a marked-block edit keeps the surrounding factory intact."

Type: Follow the action above in app.py; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Run and check

```
python management.py runserver
# Exercise the task URLs shown in this chapter with your cookie and new CSRF token.
```

Exercise the task API now

Use the cookie and current CSRF token from chapter 4; replace PROJECT_ID with the ID returned by chapter 5. Then read the task list and change TASK_ID to the returned task ID:

```
curl -b cookies.txt -H 'Content-Type: application/json' -H 'X-CSRF-Token: TOKEN' -d
    '{"title":"Record the lesson","status":"todo","due_date":"2026-12-01"}'
    http://127.0.0.1:8000/api/tasks/project/PROJECT_ID
curl -b cookies.txt http://127.0.0.1:8000/api/tasks/project/PROJECT_ID
curl -X PATCH -b cookies.txt -H 'Content-Type: application/json' -H 'X-CSRF-Token: TOKEN' -d
    '{"status":"done"}' http://127.0.0.1:8000/api/tasks/TASK_ID
curl -b cookies.txt 'http://127.0.0.1:8000/api/tasks/project/PROJECT_ID?status=done'
```

Expected result

A signed-in customer can create a task, mark it done, filter by done and remove it. Invalid status and impossible date inputs return 400.

Common errors

Use /api/tasks/project/PROJECT_ID for task lists and creation. Use /api/tasks/TASK_ID for PATCH and DELETE. An empty PATCH and an unknown status are rejected.

Short exercise

Create two customers and prove one cannot update the other's task.

What to say and show

Say: "Add tasks, update status and due dates, filter tasks and delete them. The server remains responsible for persistence and authorization; the browser presents the result."

Show the expected result above, then point to the file responsible for it. Run the chapter check before the next lesson.

Save your checkpoint

```
git add .
git commit -m "Complete chapter 06: Task workflow"
```

07 | Backend tests

Goal

Run realistic HTTP requests against disposable ORM databases before writing the UI.

What you are building

conftest.py creates a test app and owns its event loop and ORM lifecycle. generate_schemas is used ONLY to prepare disposable test databases; production and development use migrations. TestClient exercises the ASGI app with HTTPX. Account carries its cookie and rotated CSRF token. Tests cover valid workflows, rejected input, expired sessions and cross-user access. Read an ownership test aloud: arrange two accounts, create a project, attempt another user's request and assert it is denied.

Build this chapter

Stop the server. Work in project_manager. Follow each CREATE, whole-file replacement, or marked-block edit below in order.

Step 1: CREATE - make parent folders, then create this file - tests/conftest.py

Own the test database lifecycle and event loop. Never use development files in this fixture.

Create the parent directories, then save this complete file.

```
import asyncio
import pytest
from app import create_app
import httpx


class TestClient:
    def __init__(self, app):
        self.app = app
        self.loop = getattr(app, "test_loop", None)

    def __getattr__(self, method):
        def request(path, json_data=None, **kwargs):
            async def send():
                async with httpx.AsyncClient(
                    transport=httpx.ASGITransport(app=self.app),
                    base_url="http://testserver",
                ) as client:
                    return await client.request(
                        method.upper(), path, json=json_data, **kwargs
                    )

            return (
                self.loop.run_until_complete(send())
                if self.loop
                else asyncio.run(send())
            )

        return request


from settings import ROOT
```

```
@pytest.fixture
def application(tmp_path):
    path = tmp_path / "app.sqlite3"
    app = create_app(path, tmp_path / "admin.sqlite3", debug=True)
    loop = asyncio.new_event_loop()
    app.test_loop = loop
    loop.run_until_complete(app.db.initialize())
    with app.db.bind():
        loop.run_until_complete(app.db.context.generate_schemas())
    yield app
    loop.run_until_complete(app.db.close())
    loop.close()


@pytest.fixture
def client(application):
    return TestClient(application)


class Account:
    def __init__(self, client, email="learner@example.test"):
        self.client = client
        response = client.get("/api/auth/session")
        self.accept_session(response)
        response = self.call(
            "/api/auth/register",
            "post",
            {"name": "Learner", "email": email, "password": "LearningPython123!"},
        )
        assert response.status_code == 201, response.content
        self.accept_session(response)

    def accept_session(self, response):
        self.cookie_header = next(

            value
            for value in response.headers.get_list("set-cookie")
            if value.startswith("project_session=")
        )
        self.cookie = self.cookie_header.split(";")[0]
        self.csrf = response.json()["data"]["csrf"]

    @property
    def headers(self):
        return {
            "cookie": self.cookie,
            "x-csrf-token": self.csrf,
            "content-type": "application/json",
        }

    def call(self, path, method="get", data=None, headers=None):
        options = {"headers": self.headers if headers is None else headers}
        if data is not None:
            options["json_data"] = data
        return getattr(self.client, method)(path, **options)


@pytest.fixture
def account(client):
    return Account(client)


def orm(application, query):
    """Run a direct ORM assertion in the same database context as this test app."""
    with application.db.bind():
```

```
        return application.test_loop.run_until_complete(query())
```

Say: "Own the test database lifecycle and event loop. Never use development files in this fixture."

Type: Follow the action above in `tests/conftest.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 2: CREATE - make parent folders, then create this file - tests/test_api.py

This file owns the feature named by its module or component. Read the route/form flow before continuing.

Create the parent directories, then save this complete file.

```
import asyncio
import time
import pytest
from conftest import Account, orm
from models import Task, Session


def project(account):
    response = account.call(
        "/api/projects/",
        "post",
        {"name": "Course project", "description": "Build the app"},
    )
    assert response.status_code == 201
    return response.json()["data"]


def test_registration_rotates_session_and_never_returns_password_hash(client, account):
    response = account.call("/api/auth/session")
    assert response.status_code == 200
    assert response.json()["data"]["user"]["email"] == "learner@example.test"
    assert "password" not in str(response.json())
    assert "httponly" in account.cookie_header.lower()
    assert "samesite=lax" in account.cookie_header.lower()


def test_login_logout_and_old_session_revocation(account):
    old_headers = account.headers
    response = account.call("/api/auth/logout", "post", {})
    assert response.status_code == 200
    account.accept_session(response)
    assert account.call("/api/projects/", headers=old_headers).status_code == 401
    response = account.call(
        "/api/auth/login",

        "post",
        {"email": "learner@example.test", "password": "LearningPython123!"},
    )
    assert response.status_code == 200
    account.accept_session(response)
    assert account.call("/api/projects/").status_code == 200


@pytest.mark.parametrize(
    "payload",
```

```
    [
        [],
        {"name": "", "email": "bad", "password": "short"},
        {"name": "A", "email": "a@test.test", "password": "short"},
    ],
)
def test_invalid_registration(account, payload):
    assert account.call("/api/auth/register", "post", payload).status_code == 400


def test_wrong_password_and_duplicate_registration(account):
    assert (
        account.call(
            "/api/auth/login",
            "post",
            {"email": "learner@example.test", "password": "IncorrectPassword123!"},
        ).status_code
        == 401
    )
    assert (
        account.call(
            "/api/auth/register",
            "post",
            {

                "name": "Other",
                "email": "LEARNER@example.test",
                "password": "LearningPython123!",
            },
        ).status_code
        == 409
    )


def test_authentication_and_csrf_boundaries(client, account):
    assert client.get("/api/projects/").status_code == 401
    assert (
        account.call(
            "/api/projects/",
            "post",
            {"name": "Blocked"},
            headers={"cookie": account.cookie},
        ).status_code
        == 403
    )
    other = Account(client, "other@example.test")
    headers = {**account.headers, "x-csrf-token": other.csrf}
    assert (
        account.call(
            "/api/projects/", "post", {"name": "Blocked"}, headers=headers
        ).status_code
        == 403
    )
    headers = {**account.headers, "origin": "https://evil.example"}
    assert (
        account.call(
            "/api/projects/", "post", {"name": "Blocked"}, headers=headers
        ).status_code
        == 403

    )


def test_project_crud_and_ownership(client, account):
```

```
    item = project(account)
    other = Account(client, "other@example.test")
    path = f"/api/projects/{item['id']}"
    for method in ["get", "put", "delete"]:
        assert (
            other.call(
                path, method, {"name": "Stolen"} if method != "get" else None
            ).status_code
            == 404
        )
    assert other.call("/api/projects/").json()["data"] == []
    assert (
        account.call(path, "put", {"name": "Renamed", "description": "Updated"}).json()[
            "data"
        ]["name"]
        == "Renamed"
    )
    assert account.call(path, "delete", {}).status_code == 200
    assert account.call(path).status_code == 404


def test_task_workflow_filter_and_cascade(account, application):
    item = project(account)
    path = f"/api/tasks/project/{item['id']}"
    response = account.call(
        path, "post", {"title": "Record lesson", "due_date": "2026-12-01"}
    )
    assert response.status_code == 201
    task_id = response.json()["data"]["id"]
    assert (

        account.call(f"/api/tasks/{task_id}", "patch", {"status": "done"}).status_code
        == 200
    )
    assert len(account.call(path + "?status=done").json()["data"]) == 1
    assert account.call(path + "?status=todo").json()["data"] == []
    assert account.call(path + "?status=unknown").status_code == 400
    account.call(f"/api/projects/{item['id']}", "delete", {})
    assert orm(application, lambda: Task.filter(id=task_id).first()) is None


@pytest.mark.parametrize(
    "data",
    [
        {"title": ""},
        {"title": "Task", "status": "invalid"},
        {"title": "Task", "due_date": "2026-02-30"},
        {"title": 123},
    ],
)
def test_task_validation(account, data):
    item = project(account)
    assert (
        account.call(f"/api/tasks/project/{item['id']}", "post", data).status_code
        == 400
    )


def test_task_ownership_and_deletion(client, account):
    item = project(account)
    path = f"/api/tasks/project/{item['id']}"
    task = account.call(path, "post", {"title": "Private task"}).json()["data"]
    other = Account(client, "other@example.test")
    assert other.call(path).status_code == 404
    assert other.call(path, "post", {"title": "Wrong owner"}).status_code == 404
```

```
    assert (
        other.call(f"/api/tasks/{task['id']}", "patch", {"status": "done"}).status_code
        == 404
    )
    assert other.call(f"/api/tasks/{task['id']}", "delete", {}).status_code == 404
    assert account.call(f"/api/tasks/{task['id']}", "delete", {}).status_code == 200


def test_expired_session_is_rejected(account, application):
    orm(application, lambda: Session.all().update(expires_at=int(time.time()) - 1))
    assert account.call("/api/projects/").status_code == 401


def test_sql_values_are_not_executed(account):
    response = account.call(
        "/api/projects/", "post", {"name": "Robert'); DROP TABLE users;--"}
    )
    assert response.status_code == 201
    assert account.call("/api/auth/session").json()["data"]["user"] is not None


def test_auth_rate_limit(account):
    codes = [
        account.call(
            "/api/auth/login",
            "post",
            {"email": "learner@example.test", "password": "WrongPassword123!"},
        ).status_code
        for _ in range(11)
    ]
    assert 429 in codes


def test_unknown_api_is_404(client):

    assert client.get("/api/not-a-route").status_code == 404


def test_request_body_limit(client):
    response = client.post("/api/auth/register", json_data={"name": "x" * (300 * 1024)})
    assert response.status_code == 413


def test_production_requires_secret(monkeypatch, tmp_path):
    from app import create_app

    monkeypatch.delenv("FLAXON_SECRET_KEY", raising=False)
    with pytest.raises(ValueError, match="FLAXON_SECRET_KEY"):
        create_app(tmp_path / "app.sqlite3", tmp_path / "admin.sqlite3", debug=False)
```

Say: "This file owns the feature named by its module or component. Read the route/form flow before continuing."

Type: Follow the action above in `tests/test_api.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Run and check

```
python -m pytest -q tests/test_api.py
```

Expected result

tests/test_api.py passes. Each test uses temporary files and does not modify your learner database.

Common errors

Do not run later Admin tests yet. Keep the ORM context and event loop alive for the whole fixture. Sharing one account's cookie with another makes an ownership test invalid.

Short exercise

Add a regression test that sends an extra owner_id in project JSON and proves it cannot choose a different owner.

What to say and show

Say: "Run realistic HTTP requests against disposable ORM databases before writing the UI. The server remains responsible for persistence and authorization; the browser presents the result."

Show the expected result above, then point to the file responsible for it. Run the chapter check before the next lesson.

Save your checkpoint

```
git add .
git commit -m "Complete chapter 07: Backend tests"
```

08 | Teloce HTML shell and scoped CSS

Goal

Serve a browser shell and placeholder pages from the existing backend.

What you are building

app.use_teloce registers the compiler, runtime and browser routes. request.compile returns the compiled app.html shell for known page URLs. Each .html file has a template, script lang=ts and style scoped. The shell provides data-teloce-router-view. Module ui_routes maps compiled page names to browser paths. The first account/project pages are labelled placeholders; we replace them in the next two chapters. Shared types.ts describes JSON contracts and api.ts includes cookies, CSRF and error handling. Scoped CSS belongs to each component, so there is no separate app.css file.

Build this chapter

Stop the server. Work in project_manager. Follow each CREATE, whole-file replacement, or marked-block edit below in order.

Step 1: CREATE - make parent folders, then create this file - ui/types.ts

Match the Python response fields exactly. These types document the browser contract; runtime validation stays on the server.

Create the parent directories, then save this complete file.

```
export interface User {
  id: number;
  name: string;
  email: string;
}
export interface Project {
  id: number;
  owner_id: number;
  name: string;
  description: string;
}
export type TaskStatus = "todo" | "doing" | "done";
export interface Task {
  id: number;
  project_id: number;
  title: string;
  status: TaskStatus;
  due_date: string | null;
}
export interface Session {
  user: User | null;
  csrf: string;
}
export interface Article {
  id: string;
  title: string;
  slug: string;
  summary: string;
  body?: string;
}
```

Say: "Match the Python response fields exactly. These types document the browser contract; runtime validation stays on the server."

Type: Follow the action above in `ui/types.ts`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 2: CREATE - make parent folders, then create this file - ui/api.ts

Centralize fetch, cookies, CSRF rotation and error messages here so pages share one contract.

Create the parent directories, then save this complete file.

```
import type { Session } from "./types";

let session: Session | null = null;
export async function loadSession(): Promise<Session> {
  const response = await fetch("/api/auth/session", { credentials: "same-origin" });
  const payload = await response.json();
  if (!response.ok)
    throw new Error(payload.error?.message || "Could not load your session.");
  session = payload.data;
  return session!;
}

export async function api<T>(path: string, method = "GET", body?: unknown): Promise<T> {
  if (!session) await loadSession();
  const response = await fetch(path, {
    method,
    credentials: "same-origin",
    headers: { "Content-Type": "application/json", "X-CSRF-Token": session!.csrf },
    body: body === undefined ? undefined : JSON.stringify(body),
  });
  const payload = await response.json();
  if (!response.ok) {
    if (response.status === 401 || response.status === 403) session = null;
    throw new Error(payload.error?.message || "The request failed. Please try again.");
  }
  if (path.startsWith("/api/auth/")) session = payload.data;
  return payload.data as T;
}
```

Say: "Centralize fetch, cookies, CSRF rotation and error messages here so pages share one contract."

Type: Follow the action above in `ui/api.ts`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 3: EDIT - replace the entire file - ui/app.html

This is the SPA shell. Its internal anchors and router view cooperate with each module's page mappings.

Select all existing contents of this file and replace them with the complete code below.

```
<template>
  <div class="app-shell">
    <header>
      <a href="/" data-teloce-link class="brand">Project Manager</a>
      <nav aria-label="Main navigation">
        <a href="/projects" data-teloce-link>Projects</a>
```

```
        <a href="/login" data-teloce-link>Account</a>
      </nav>
    </header>
    <main id="router-view" data-teloce-router-view></main>
    <footer>Built with Flaxon and Teloce. <a href="/admin/">Staff Admin</a></footer>
  </div>
</template>
<script lang="ts">
export default {};
</script>
<style scoped>
/* Only the document reset is global: body is outside this component. */
:global(body) {
  margin: 0;
}
.app-shell {
  min-height: 100vh;
  font-family: system-ui, sans-serif;
  color: #17243a;
  background: #f3f6fc;
}
header {
  display: flex;
  justify-content: space-between;
  gap: 1rem;
  align-items: center;
  padding: 1.2rem max(1rem, calc((100vw - 1080px) / 2));
  background: #17243a;

  color: white;
}
header a {
  color: white;
  text-decoration: none;
}
.brand {
  font-weight: 800;
  font-size: 1.3rem;
}
nav {
  display: flex;
  gap: 1rem;
  flex-wrap: wrap;
}
main {
  max-width: 1080px;
  margin: auto;
  min-height: 75vh;
  padding: 2rem 1rem;
}
footer {
  padding: 1.5rem;
  text-align: center;
  color: #526079;
}
footer a {
  color: #1d4ed8;
}
:focus-visible {
  outline: 3px solid #f59e0b;
  outline-offset: 3px;
}
@media (max-width: 520px) {
```

```
  header {
    flex-direction: column;
    align-items: flex-start;
  }
  main {
    padding: 1rem;
  }
}
</style>
```

Say: "This is the SPA shell. Its internal anchors and router view cooperate with each module's page mappings."

Type: Follow the action above in `ui/app.html`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 4: CREATE - make parent folders, then create this file - ui/pages/Home.html

This file owns the feature named by its module or component. Read the route/form flow before continuing.

Create the parent directories, then save this complete file.

```
<template>
  <section class="hero">
    <p class="eyebrow">A practical Python full-stack course</p>
    <h1>Make room for your next idea.</h1>
    <p>Create projects, break them into tasks, and watch your progress.</p>
    <a class="button" href="/projects" data-teloce-link>Open projects</a>
    <a href="/login" data-teloce-link>Create an account</a>
  </section>
</template>
<script lang="ts">
export default {};
</script>
<style scoped>
* {
  box-sizing: border-box;
}
a {
  color: #1d4ed8;
}
h1 {
  font-size: clamp(1.8rem, 5vw, 3rem);
  line-height: 1.15;
}
h2 {
  font-size: 1.2rem;
}
p {
  line-height: 1.6;
}
[role="alert"] {
  color: #a21c2b;
  padding: 0.8rem;
  border-left: 4px solid;
}

:focus-visible {
  outline: 3px solid #f59e0b;
```

```
  outline-offset: 3px;
}
.hero {
  max-width: 700px;
  padding: 3rem 0;
}
.eyebrow {
  text-transform: uppercase;
  font-size: 0.8rem;
  letter-spacing: 0.1em;
}
.button {
  display: inline-block;
  border-radius: 7px;
  min-height: 44px;
  background: #2563eb;
  color: white;
  padding: 0.7rem 1rem;
  text-decoration: none;
  margin-right: 0.5rem;
}
</style>
```

Say: "This file owns the feature named by its module or component. Read the route/form flow before continuing."

Type: Follow the action above in `ui/pages/Home.html`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 5: CREATE - make parent folders, then create this file - modules/auth/ui/pages/Login.html

Replace the placeholder with account forms. Use the session API and rotated CSRF token; do not read the HttpOnly cookie from JavaScript.

Create the parent directories, then save this complete file.

```
<template><section><h1>Account</h1><p>This screen is built in the next UI
    lesson.</p></section></template>
<script lang="ts">export default {};</script>
<style scoped>
h1 { font-size: 2rem; }
p { line-height: 1.6; }
</style>
```

Say: "Replace the placeholder with account forms. Use the session API and rotated CSRF token; do not read the HttpOnly cookie from JavaScript."

Type: Follow the action above in `modules/auth/ui/pages/Login.html`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 6: CREATE - make parent folders, then create this file - modules/projects/ui/pages/ProjectList.html

Replace the placeholder with the complete projects screen. Connect its form to the typed API helper; show loading, error and empty states. Keep all CSS inside style scoped.

Create the parent directories, then save this complete file.

```
<template><section><h1>Projects</h1><p>This screen is built in the next UI
    lesson.</p></section></template>
<script lang="ts">export default {};</script>
<style scoped>
h1 { font-size: 2rem; }
p { line-height: 1.6; }
</style>
```

Say: "Replace the placeholder with the complete projects screen. Connect its form to the typed API helper; show loading, error and empty states. Keep all CSS inside style scoped."

Type: Follow the action above in `modules/projects/ui/pages/ProjectList.html`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 7: CREATE - make parent folders, then create this file - modules/projects/ui/pages/ProjectDetails/[id].html

Replace the placeholder with the project detail page. Load the owned project and its tasks using the router ID, then compose the task form, list and progress display.

Create the parent directories, then save this complete file.

```
<template><section><h1>Project details</h1><p>This screen is built in the next UI
    lesson.</p></section></template>
<script lang="ts">export default {props: ['id']};</script>
<style scoped>
h1 { font-size: 2rem; }
p { line-height: 1.6; }
</style>
```

Say: "Replace the placeholder with the project detail page. Load the owned project and its tasks using the router ID, then compose the task form, list and progress display."

Type: Follow the action above in `modules/projects/ui/pages/ProjectDetails/[id].html`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 8: EDIT - replace the marked blocks - app.py

The application factory mounts only the features already introduced. Follow the action label: a marked-block edit keeps the surrounding factory intact.

Edit 1: find this exact block

Open `app.py`. Locate this block once. The unchanged surrounding lines identify its position; do not use line numbers from another revision.

```
    if app.is_management:
        return app


    @app.get("/health/course")
    async def health():
```

Replace that block with

```
    if app.is_management:
        return app

    app.use_teloce(
        project_root=ROOT,
        ui_dir="ui",
        title="Project Manager",
        options={"minifier": "minifyjs"},
    )

    @app.get("/health/course")
    async def health():
```

Leave the rest of the file unchanged.

Edit 2: find this exact block

Open `app.py`. Locate this block once. The unchanged surrounding lines identify its position; do not use line numbers from another revision.

```
        await User.all().limit(1)
        return {"data": {"status": "ok"}}

    @app.get("/")
    async def home():
        return {"data": {"message": "Backend lesson ready. The SPA starts in chapter 8."}}

    return app
```

Replace that block with

```
        await User.all().limit(1)
        return {"data": {"status": "ok"}}

    # Explicit shell routes preserve real 404 responses for unknown API paths.
    @app.get("/")
    @app.get("/login")
    @app.get("/projects")
    async def spa(request: Request, project_id=None, slug=None):
        return await request.compile("app.html", {})

    return app
```

Leave the rest of the file unchanged.

Say: "The application factory mounts only the features already introduced. Follow the action label: a marked-block edit keeps the surrounding factory intact."

Type: Follow the action above in `app.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Remove public/app.css

The replacement factory and components no longer load this generated starter stylesheet. Remove the file after replacing the shell.

Run and check

```
python management.py runserver
# Open http://127.0.0.1:8000/
# Account/project screens are placeholders until chapter 9.
```

Expected result

The shell and navigation appear at /. The account and project placeholder pages mount. /api/welcome/status still responds as JSON.

Common errors

A blank page can be a compiler/import error: inspect terminal and browser console. Delete public/app.css after replacing the factory; the completed app does not mount it. The shell's explicit global body reset is intentional; child components own their other rules.

Short exercise

Change the shell's scoped header rules without modifying a child component's styles.

What to say and show

Say: "Serve a browser shell and placeholder pages from the existing backend. The server remains responsible for persistence and authorization; the browser presents the result."

Show the expected result above, then point to the file responsible for it. Run the chapter check before the next lesson.

Save your checkpoint

```
git add .
git commit -m "Complete chapter 08: Teloce HTML shell and scoped CSS"
```

09 | Login and project screens

Goal

Replace placeholders with working account forms and a project list.

What you are building

Login uses the shared API helper to load the anonymous session, submit register/login, and replace cached session state after rotation. Loading guards disable duplicate submissions and error messages remain visible. ProjectList requests only authorized records from the API and creates a project from a form. Internal project anchors use data-teloce-link. The server owns authorization; TypeScript types and disabled buttons do not enforce permissions.

Build this chapter

Stop the server. Work in project_manager. Follow each CREATE, whole-file replacement, or marked-block edit below in order.

Step 1: EDIT - replace the entire file - modules/auth/ui/pages/Login.html

Replace the placeholder with account forms. Use the session API and rotated CSRF token; do not read the HttpOnly cookie from JavaScript.

Select all existing contents of this file and replace them with the complete code below.

```
<template>
  <section class="narrow">
    <h1>Your account</h1>
    <p v-if="busy" role="status">Please wait...</p>
    <p v-if="message" role="alert">{{ message }}</p>
    <div v-if="user">
      <p>Signed in as {{ user.name }}.</p>
      <a href="/projects" data-teloce-link>Open projects</a>
      <button type="button" @click="logout" :disabled="busy">Sign out</button>
    </div>
    <form v-show="!user" @submit.prevent="submit">
      <label v-show="registering"
        >Name<input
          name="name"
          v-model="name"
          maxlength="80"
          :required="registering"
          autocomplete="name"
      /></label>
      <label
        >Email<input
          name="email"
          type="email"
          v-model="email"
          required
          autocomplete="username"
      /></label>
      <label
        >Password<input
          name="password"
          type="password"
          v-model="password"
```

```
          minlength="12"
          maxlength="128"

          required
          autocomplete="current-password"
      /></label>
      <button :disabled="busy">{{ registering ? "Create account" : "Sign in" }}</button>
      <button
        type="button"
        class="secondary"
        @click="registering = !registering"
        :disabled="busy"
      >
        {{ registering ? "Use an existing account" : "Register instead" }}
      </button>
    </form>
  </section>
</template>
<script lang="ts">
import { api, loadSession } from "../../../../ui/api.ts";
import type { Session } from "../../../../ui/types.ts";
export default {
  data() {
    return {
      user: null,
      registering: false,
      name: "",
      email: "",
      password: "",
      busy: false,
      message: "",
    };
  },
  async mounted() {
    try {
      this.user = (await loadSession()).user;
    } catch (error) {

      this.message = error.message;
    }
  },
  methods: {
    async submit() {
      if (this.busy) return;
      this.busy = true;
      this.message = "";
      try {
        const path = this.registering ? "/api/auth/register" : "/api/auth/login";
        const result = await api<Session>(path, "POST", {
          name: this.name,
          email: this.email,
          password: this.password,
        });
        this.user = result.user;
        this.password = "";
        window.__TELOCE_ROUTER__.navigate("/projects");
      } catch (error) {
        this.message = error.message;
      } finally {
        this.busy = false;
      }
    },
    async logout() {
      this.busy = true;
```

```
      try {
        await api("/api/auth/logout", "POST", {});
        this.user = null;
        this.message = "Signed out.";
      } catch (error) {
        this.message = error.message;
      } finally {
        this.busy = false;

      }
    },
  },
};
</script>
<style scoped>
* {
  box-sizing: border-box;
}
a {
  color: #1d4ed8;
}
h1 {
  font-size: clamp(1.8rem, 5vw, 3rem);
  line-height: 1.15;
}
h2 {
  font-size: 1.2rem;
}
p {
  line-height: 1.6;
}
[role="alert"] {
  color: #a21c2b;
  padding: 0.8rem;
  border-left: 4px solid;
}
:focus-visible {
  outline: 3px solid #f59e0b;
  outline-offset: 3px;
}
label {
  display: grid;
  gap: 0.4rem;

  margin: 0.9rem 0;
  font-weight: 600;
}
input,
select,
textarea,
button {
  font: inherit;
  border-radius: 7px;
  padding: 0.7rem;
  min-height: 44px;
}
input,
select,
textarea {
  width: 100%;
  border: 1px solid #abb9cf;
  background: white;
  color: #17243a;
}
```

```
button {
  display: inline-block;
  border: 0;
  background: #2563eb;
  color: white;
  padding: 0.7rem 1rem;
  cursor: pointer;
  margin: 0.3rem 0.5rem 0.3rem 0;
}
button:disabled {
  opacity: 0.55;
  cursor: wait;
}
.secondary {

  background: #e4eaf5;
  color: #17243a;
}
.danger {
  background: #b42332;
}
.narrow {
  max-width: 480px;
  margin: auto;
}
</style>
```

Say: "Replace the placeholder with account forms. Use the session API and rotated CSRF token; do not read the HttpOnly cookie from JavaScript."

Type: Follow the action above in `modules/auth/ui/pages/Login.html`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 2: EDIT - replace the entire file - modules/projects/ui/pages/ProjectList.html

Replace the placeholder with the complete projects screen. Connect its form to the typed API helper; show loading, error and empty states. Keep all CSS inside style scoped.

Select all existing contents of this file and replace them with the complete code below.

```
<template>
  <section>
    <h1>Your projects</h1>
    <p>Choose a project to manage its tasks.</p>
    <p v-if="message" role="alert">
      {{ message }} <a href="/login" data-teloce-link>Account</a>
    </p>
    <p v-if="loading" role="status">Loading projects...</p>
    <form @submit.prevent="createProject" class="card">
      <h2>Create a project</h2>
      <label
        >Name<input name="project-name" v-model="name" required maxlength="120"
      /></label>
      <label
        >Description<textarea v-model="description" maxlength="1000"></textarea>
      </label>
      <button :disabled="busy">{{ busy ? "Saving..." : "Create project" }}</button>
    </form>
```

```
    <p v-if="!loading && projects.length === 0">
      No projects yet. Create your first one above.
    </p>
    <div class="cards">
      <a
        v-for="project in projects"
        :key="project.id"
        :href="'/projects/' + project.id"
        data-teloce-link
        class="card"
      >
        <h2>{{ project.name }}</h2>
        <p>{{ project.description }}</p>
      </a>
    </div>
  </section>

</template>
<script lang="ts">
import { api } from "../../../../ui/api.ts";
import type { Project } from "../../../../ui/types.ts";
export default {
  data() {
    return {
      projects: [],
      name: "",
      description: "",
      message: "",
      loading: true,
      busy: false,
    };
  },
  async mounted() {
    try {
      this.projects = await api<Project[]>("/api/projects/");
    } catch (error) {
      this.message = error.message;
    } finally {
      this.loading = false;
    }
  },
  methods: {
    async createProject() {
      if (this.busy) return;
      this.busy = true;
      this.message = "";
      try {
        const project = await api<Project>("/api/projects/", "POST", {
          name: this.name,
          description: this.description,
        });

        this.projects = [project, ...this.projects];
        this.name = "";
        this.description = "";
      } catch (error) {
        this.message = error.message;
      } finally {
        this.busy = false;
      }
    },
  },
};
</script>
```

```
<style scoped>
* {
  box-sizing: border-box;
}
a {
  color: #1d4ed8;
}
h1 {
  font-size: clamp(1.8rem, 5vw, 3rem);
  line-height: 1.15;
}
h2 {
  font-size: 1.2rem;
}
p {
  line-height: 1.6;
}
[role="alert"] {
  color: #a21c2b;
  padding: 0.8rem;
  border-left: 4px solid;
}

:focus-visible {
  outline: 3px solid #f59e0b;
  outline-offset: 3px;
}
.card {
  display: block;
  background: white;
  border: 1px solid #d8e0ed;
  border-radius: 12px;
  padding: 1.2rem;
  margin: 1rem 0;
  color: inherit;
  text-decoration: none;
}
label {
  display: grid;
  gap: 0.4rem;
  margin: 0.9rem 0;
  font-weight: 600;
}
input,
select,
textarea,
button {
  font: inherit;
  border-radius: 7px;
  padding: 0.7rem;
  min-height: 44px;
}
input,
select,
textarea {
  width: 100%;
  border: 1px solid #abb9cf;

  background: white;
  color: #17243a;
}
button {
  display: inline-block;
  border: 0;
```

```
  background: #2563eb;
  color: white;
  padding: 0.7rem 1rem;
  cursor: pointer;
  margin: 0.3rem 0.5rem 0.3rem 0;
}
button:disabled {
  opacity: 0.55;
  cursor: wait;
}
.secondary {
  background: #e4eaf5;
  color: #17243a;
}
.danger {
  background: #b42332;
}
.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(min(100%, 240px), 1fr));
  gap: 1rem;
}
</style>
```

Say: "Replace the placeholder with the complete projects screen. Connect its form to the typed API helper; show loading, error and empty states. Keep all CSS inside style scoped."

Type: Follow the action above in `modules/projects/ui/pages/ProjectList.html`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Run and check

```
python management.py runserver
# Open /login; register; create a project on /projects.
# Project details are completed in chapter 10.
```

Expected result

Register in /login, open /projects and create a project. Sign out and sign back in. The new project survives the restart.

Common errors

Always inspect response.ok. A resolved fetch promise can still represent a 400/403/500. Relative imports are resolved from each HTML component. Registration must refresh the cached CSRF token.

Short exercise

Show the validation message for an empty project name and make the same denied request directly to the API.

What to say and show

Say: "Replace placeholders with working account forms and a project list. The server remains responsible for persistence and authorization; the browser presents the result."

Show the expected result above, then point to the file responsible for it. Run the chapter check before the next lesson.

Save your checkpoint

```
git add .
git commit -m "Complete chapter 09: Login and project screens"
```

10 | Task components, signals and progress

Goal

Complete the task detail screen using reusable components and reactive progress.

What you are building

TaskForm emits a create event; TaskList emits status and remove events. ProjectDetails owns API calls and the task collection. signal stores that collection, computed derives completion, and an effect copies the value into displayed component data. Stop the effect before unmount. Teloce's .html compiler imports detected signal helpers automatically: do not add a redundant import. Ordinary TypeScript modules do not get the same automatic imports. Replacing the collection via its setter keeps progress and task display consistent. Empty projects return 0 percent.

Build this chapter

Stop the server. Work in project_manager. Follow each CREATE, whole-file replacement, or marked-block edit below in order.

Step 1: CREATE - make parent folders, then create this file - modules/projects/ui/components/TaskForm.html

Create the reusable task form. Collect title, status and due date, validate feedback and emit the saved task to its parent.

Create the parent directories, then save this complete file.

```
<template>
  <form @submit.prevent="submit" class="card">
    <h2>Add a task</h2>
    <label
      >Task title<input name="task-title" v-model="title" maxlength="160" required
    /></label>
    <label>Due date<input name="due-date" type="date" v-model="dueDate" /></label>
    <button :disabled="busy">Add task</button>
  </form>
</template>
<script lang="ts">
export default {
  props: ["busy"],
  emits: ["create"],
  data() {
    return { title: "", dueDate: "" };
  },
  methods: {
```

```
    submit() {
      if (this.busy || !this.title.trim()) return;
      this.$emit("create", { title: this.title, due_date: this.dueDate || null });
    },
  },
};
</script>
<style scoped>
* {
  box-sizing: border-box;
}
a {
  color: #1d4ed8;
}
h1 {
  font-size: clamp(1.8rem, 5vw, 3rem);

  line-height: 1.15;
}
h2 {
  font-size: 1.2rem;
}
p {
  line-height: 1.6;
}
[role="alert"] {
  color: #a21c2b;
  padding: 0.8rem;
  border-left: 4px solid;
}
:focus-visible {
  outline: 3px solid #f59e0b;
  outline-offset: 3px;
}
.card {
  display: block;
  background: white;
  border: 1px solid #d8e0ed;
  border-radius: 12px;
  padding: 1.2rem;
  margin: 1rem 0;
  color: inherit;
  text-decoration: none;
}
label {
  display: grid;
  gap: 0.4rem;
  margin: 0.9rem 0;
  font-weight: 600;
}
input,

select,
textarea,
button {
  font: inherit;
  border-radius: 7px;
  padding: 0.7rem;
  min-height: 44px;
}
input,
select,
textarea {
  width: 100%;
```

```
  border: 1px solid #abb9cf;
  background: white;
  color: #17243a;
}
button {
  display: inline-block;
  border: 0;
  background: #2563eb;
  color: white;
  padding: 0.7rem 1rem;
  cursor: pointer;
  margin: 0.3rem 0.5rem 0.3rem 0;
}
button:disabled {
  opacity: 0.55;
  cursor: wait;
}
.secondary {
  background: #e4eaf5;
  color: #17243a;
}
.danger {

  background: #b42332;
}
</style>
```

Say: "Create the reusable task form. Collect title, status and due date, validate feedback and emit the saved task to its parent."

Type: Follow the action above in `modules/projects/ui/components/TaskForm.html`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 2: CREATE - make parent folders, then create this file - modules/projects/ui/components/TaskList.html

Create the reusable task list. Use stable task IDs for keyed rows and show the filtered workflow without changing database ownership.

Create the parent directories, then save this complete file.

```
<template>
  <div>
    <p v-if="tasks.length === 0">No tasks match this filter.</p>
    <ul class="task-list">
      <li v-for="task in tasks" :key="task.id" class="card">
        <strong>{{ task.title }}</strong
        ><span>{{ task.due_date ? "Due " + task.due_date : "No due date" }}</span>
        <label
          >Status<select
            aria-label="Task status"
            :value="task.status"
            @change="$emit('status', { id: task.id, status: $event.target.value })"
            :disabled="busy"
          >
            <option value="todo">To do</option>
            <option value="doing">Doing</option>
            <option value="done">Done</option>
          </select></label
```

```
        >
        <button
          class="danger"
          type="button"
          @click="$emit('remove', task.id)"
          :disabled="busy"
        >
          Delete task
        </button>
      </li>
    </ul>
  </div>
</template>
<script lang="ts">
export default { props: ["tasks", "busy"], emits: ["status", "remove"] };
</script>

<style scoped>
* {
  box-sizing: border-box;
}
a {
  color: #1d4ed8;
}
h1 {
  font-size: clamp(1.8rem, 5vw, 3rem);
  line-height: 1.15;
}
h2 {
  font-size: 1.2rem;
}
p {
  line-height: 1.6;
}
[role="alert"] {
  color: #a21c2b;
  padding: 0.8rem;
  border-left: 4px solid;
}
:focus-visible {
  outline: 3px solid #f59e0b;
  outline-offset: 3px;
}
.card {
  display: block;
  background: white;
  border: 1px solid #d8e0ed;
  border-radius: 12px;
  padding: 1.2rem;
  margin: 1rem 0;
  color: inherit;

  text-decoration: none;
}
label {
  display: grid;
  gap: 0.4rem;
  margin: 0.9rem 0;
  font-weight: 600;
}
input,
select,
textarea,
button {
```

```
  font: inherit;
  border-radius: 7px;
  padding: 0.7rem;
  min-height: 44px;
}
input,
select,
textarea {
  width: 100%;
  border: 1px solid #abb9cf;
  background: white;
  color: #17243a;
}
button {
  display: inline-block;
  border: 0;
  background: #2563eb;
  color: white;
  padding: 0.7rem 1rem;
  cursor: pointer;
  margin: 0.3rem 0.5rem 0.3rem 0;
}

button:disabled {
  opacity: 0.55;
  cursor: wait;
}
.secondary {
  background: #e4eaf5;
  color: #17243a;
}
.danger {
  background: #b42332;
}
.task-list {
  list-style: none;
  padding: 0;
}
.task-list span {
  display: block;
  color: #526079;
  margin-top: 0.5rem;
}
</style>
```

Say: "Create the reusable task list. Use stable task IDs for keyed rows and show the filtered workflow without changing database ownership."

Type: Follow the action above in `modules/projects/ui/components/TaskList.html`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 3: EDIT - replace the entire file - modules/projects/ui/pages/ProjectDetails/[id].html

Replace the placeholder with the project detail page. Load the owned project and its tasks using the router ID, then compose the task form, list and progress display.

Select all existing contents of this file and replace them with the complete code below.

```
<template>
  <section>
    <a href="/projects" data-teloce-link><- All projects</a>
    <p v-if="loading" role="status">Loading project...</p>
    <p v-if="message" role="alert">{{ message }}</p>
    <div v-if="project">
      <h1>{{ project.name }}</h1>
      <p>{{ project.description }}</p>
      <p aria-live="polite">{{ progress }}% complete</p>
      <progress :value="progress" max="100" aria-label="Project progress"></progress>
      <form @submit.prevent="saveProject" class="card">
        <h2>Edit project</h2>
        <label>Name<input v-model="name" required maxlength="120" /></label>
        <label
          >Description<textarea v-model="description" maxlength="1000"></textarea>
        </label>
        <button :disabled="busy">Save project</button>
        <button type="button" class="danger" @click="deleteProject" :disabled="busy">
          Delete project
        </button>
      </form>
      <TaskForm :busy="busy" @create="createTask" />
      <label
        >Filter tasks<select v-model="filter">
          <option value="all">All tasks</option>
          <option value="todo">To do</option>
          <option value="doing">Doing</option>
          <option value="done">Done</option>
        </select></label
      >
      <TaskList
        :tasks="visibleTasks"
        :busy="busy"
        @status="changeStatus"

        @remove="removeTask"
      />
    </div>
  </section>
</template>
<script lang="ts">
import TaskForm from "../../components/TaskForm.html";
import TaskList from "../../components/TaskList.html";
import { api } from "../../../../../ui/api.ts";
import type { Project, Task } from "../../../../../ui/types.ts";

// The compiler imports these signal helpers automatically for this .html component.
const taskState = signal([]);
const completion = computed(() => {
  const tasks = taskState();
  return tasks.length
    ? Math.round(
        (tasks.filter((task) => task.status === "done").length * 100) / tasks.length,
      )
    : 0;
});
export default {
  props: ["id"],
  components: { TaskForm, TaskList },
  data() {
    return {
      project: null,
      tasks: [],
      name: "",
      description: "",
```

```
    filter: "all",
    progress: 0,
    busy: false,
    loading: true,

    message: "",
  };
},
computed: {
  visibleTasks() {
    return this.filter === "all"
      ? this.tasks
      : this.tasks.filter((task) => task.status === this.filter);
  },
},
async mounted() {
  taskState.set([]);
  this.progressEffect = effect(() => {
    this.progress = completion();
  });
  try {
    const [project, tasks] = await Promise.all([
      api<Project>("/api/projects/" + this.id),
      api<Task[]>("/api/tasks/project/" + this.id),
    ]);
    this.project = project;
    this.name = project.name;
    this.description = project.description;
    this.setTasks(tasks);
  } catch (error) {
    this.message = error.message;
  } finally {
    this.loading = false;
  }
},
beforeUnmount() {
  this.progressEffect?.stop();
},
methods: {

  setTasks(tasks) {
    this.tasks = tasks;
    taskState.set(tasks);
  },
  async mutate(operation) {
    if (this.busy) return;
    this.busy = true;
    this.message = "";
    try {
      await operation();
    } catch (error) {
      this.message = error.message;
    } finally {
      this.busy = false;
    }
  },
  async saveProject() {
    await this.mutate(async () => {
      this.project = await api<Project>("/api/projects/" + this.id, "PUT", {
        name: this.name,
        description: this.description,
      });
    });
  },
```

```
    async deleteProject() {
      if (!window.confirm("Delete this project and all its tasks?")) return;
      await this.mutate(async () => {
        await api("/api/projects/" + this.id, "DELETE", {});
        window.__TELOCE_ROUTER__.navigate("/projects");
      });
    },
    async createTask(data) {
      await this.mutate(async () => {
        const task = await api<Task>("/api/tasks/project/" + this.id, "POST", data);

        this.setTasks([task, ...this.tasks]);
      });
    },
    async changeStatus(data) {
      await this.mutate(async () => {
        const updated = await api<Task>("/api/tasks/" + data.id, "PATCH", {
          status: data.status,
        });
        this.setTasks(
          this.tasks.map((task) => (task.id === updated.id ? updated : task)),
        );
      });
    },
    async removeTask(id) {
      if (!window.confirm("Delete this task?")) return;
      await this.mutate(async () => {
        await api("/api/tasks/" + id, "DELETE", {});
        this.setTasks(this.tasks.filter((task) => task.id !== id));
      });
    },
  },
};
</script>
<style scoped>
* {
  box-sizing: border-box;
}
a {
  color: #1d4ed8;
}
h1 {
  font-size: clamp(1.8rem, 5vw, 3rem);
  line-height: 1.15;
}

h2 {
  font-size: 1.2rem;
}
p {
  line-height: 1.6;
}
[role="alert"] {
  color: #a21c2b;
  padding: 0.8rem;
  border-left: 4px solid;
}
:focus-visible {
  outline: 3px solid #f59e0b;
  outline-offset: 3px;
}
.card {
  display: block;
  background: white;
```

```
  border: 1px solid #d8e0ed;
  border-radius: 12px;
  padding: 1.2rem;
  margin: 1rem 0;
  color: inherit;
  text-decoration: none;
}
label {
  display: grid;
  gap: 0.4rem;
  margin: 0.9rem 0;
  font-weight: 600;
}
input,
select,
textarea,

button {
  font: inherit;
  border-radius: 7px;
  padding: 0.7rem;
  min-height: 44px;
}
input,
select,
textarea {
  width: 100%;
  border: 1px solid #abb9cf;
  background: white;
  color: #17243a;
}
button {
  display: inline-block;
  border: 0;
  background: #2563eb;
  color: white;
  padding: 0.7rem 1rem;
  cursor: pointer;
  margin: 0.3rem 0.5rem 0.3rem 0;
}
button:disabled {
  opacity: 0.55;
  cursor: wait;
}
.secondary {
  background: #e4eaf5;
  color: #17243a;
}
.danger {
  background: #b42332;
}

progress {
  width: 100%;
  height: 1.2rem;
  accent-color: #2563eb;
}
</style>
```

Say: "Replace the placeholder with the project detail page. Load the owned project and its tasks using the router ID, then compose the task form, list and progress display."

Type: Follow the action above in `modules/projects/ui/pages/ProjectDetails/[id].html`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Run and check

```
python management.py runserver
# Navigate to a project by its SPA link and add two tasks.
# Direct detail-page refresh is added in chapter 11.
```

Expected result

Create two tasks and complete one: progress becomes 50 percent. Filtering to done changes the visible rows while overall project progress remains 50 percent.

Common errors

Read a signal by calling it and replace it through set. An effect left alive after navigation can update an obsolete screen. Do not calculate project progress from only filtered rows.

Short exercise

Complete both tasks and expect 100 percent, then delete one and verify progress stays correct.

What to say and show

Say: "Complete the task detail screen using reusable components and reactive progress. The server remains responsible for persistence and authorization; the browser presents the result."

Show the expected result above, then point to the file responsible for it. Run the chapter check before the next lesson.

Save your checkpoint

```
git add .
git commit -m "Complete chapter 10: Task components, signals and progress"
```

11 | SPA routing and direct refresh

Goal

Support internal clicks, Back/Forward and direct nested URLs.

What you are building

There are two routers. Teloce's ui_routes maps /projects/:id to the detail component and passes id as a prop. data-teloce-link lets an ordinary anchor navigate without replacing the document; data-teloce-router-view provides the mount point. Flaxon's explicit /projects/<int:project_id> shell route handles direct visits and refreshes. The browser then fetches protected JSON. Keep Admin links as ordinary full-page navigation. Do not use a catch-all shell route that turns an unknown API path into HTML.

Build this chapter

Stop the server. Work in project_manager. Follow each CREATE, whole-file replacement, or marked-block edit below in order.

Step 1: EDIT - replace the marked blocks - app.py

The application factory mounts only the features already introduced. Follow the action label: a marked-block edit keeps the surrounding factory intact.

Edit 1: find this exact block

Open `app.py`. Locate this block once. The unchanged surrounding lines identify its position; do not use line numbers from another revision.

```
@app.get("/")
@app.get("/login")
@app.get("/projects")
async def spa(request: Request, project_id=None, slug=None):
    return await request.compile("app.html", {})
```

Replace that block with

```
@app.get("/")
@app.get("/login")
@app.get("/projects")
@app.get("/projects/<int:project_id>")
async def spa(request: Request, project_id=None, slug=None):
    return await request.compile("app.html", {})
```

Leave the rest of the file unchanged.

Say: "The application factory mounts only the features already introduced. Follow the action label: a marked-block edit keeps the surrounding factory intact."

Type: Follow the action above in `app.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Run and check

```
python management.py runserver
# Refresh a nested /projects/ID URL, then use Back and Forward.
```

Expected result

Click into a project, use Back and Forward and refresh /projects/ID. The same screen loads. /api/does-not-exist remains a JSON 404.

Common errors

A successful click with a failed refresh means the server shell route is missing. A document reload on an internal link means its data-teloce-link marker is missing. Use /projects/:id in browser routes and <int:project_id> in Python routes.

Short exercise

Put a temporary window marker in the browser console and prove an internal link preserves it.

What to say and show

Say: "Support internal clicks, Back/Forward and direct nested URLs. The server remains responsible for persistence and authorization; the browser presents the result."

Show the expected result above, then point to the file responsible for it. Run the chapter check before the next lesson.

Save your checkpoint

```
git add .
git commit -m "Complete chapter 11: SPA routing and direct refresh"
```

12 | Staff Admin

Goal

Create staff credentials and manage existing domain records with explicit permissions.

What you are building

The custom ORM adapters deliberately expose only approved project/task fields and enforce the application's ownership creation rules. Staff create customer-owned records through the customer workflow; permitted staff may inspect and edit existing records. Root admin.py describes registration for management checks; backoffice.py configures the actual custom dashboard adapters. This is an example of business-specific adapters. Flaxon also supports direct ORM registration for ordinary CRUD. Staff records use a separate durable AdminStore. setup-admin creates the first privileged staff account interactively, with no default password. Grant view/change/delete capabilities separately for ordinary staff.

Build this chapter

Stop the server. Work in project_manager. Follow each CREATE, whole-file replacement, or marked-block edit below in order.

Step 1: EDIT - replace the entire file - admin.py

Keep explicit registration in this file. At chapter 3 it is empty; chapter 12 adds the domain models.

Select all existing contents of this file and replace them with the complete code below.

```
"""Registration metadata checked by management.py; the custom backoffice uses this same
    schema."""

from models import Project, Task


def register(admin):
    admin.register(
        Project,
        name="project",
        fields=["name", "description"],
        readonly_fields=["id", "owner_id"],
        search_fields=["name"],
    )
    admin.register(
        Task,
        name="task",
        fields=["title", "status", "due_date"],
        readonly_fields=["id", "project_id"],
        list_filter=["status"],
    )
```

Say: "Keep explicit registration in this file. At chapter 3 it is empty; chapter 12 adds the domain models."

Type: Follow the action above in `admin.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 2: CREATE - make parent folders, then create this file - backoffice.py

These adapters query the same ORM models as the API and expose a deliberate staff field whitelist.

Create the parent directories, then save this complete file.

```
"""Staff model adapters reuse the domain database; CMS owns only editorial content."""

from pathlib import Path

from models import Project, Task
from flaxon.admin import AdminDashboard, AdminConfig
from flaxon.admin.registry import Registry
from flaxon.admin.cms import CMS, ContentType, CMSField
from flaxon.exceptions import BadRequest
from validation import text, task_fields


def configure_backoffice(app, admin_path, uploads_path):
    Path(admin_path).parent.mkdir(parents=True, exist_ok=True)
    admin = AdminDashboard(
        app,
        registry=Registry(),
        users=[],
        storage_path=str(admin_path),
        upload_dir=str(uploads_path),
        strict_permissions=True,
        microservices=False,
        config=AdminConfig(site_title="Project Manager Operations"),
    )

    class ProjectAdmin:
        @classmethod
        async def get_instances(cls):
            return await Project.all().order_by("-id").values()

        @classmethod
        async def get_instance(cls, object_id):
            return await Project.filter(id=object_id).first().values()

        @classmethod
        async def create_instance(cls, data):
            raise BadRequest(
                "Create projects through the application to assign an owner."
            )

        @classmethod
        async def update_instance(cls, object_id, data):
            current = await cls.get_instance(object_id)
            if current is None:
                return None
            name = text(data, "name")
            description = text(data, "description", 1000, required=False)
            await Project.filter(id=object_id).update(
                name=name, description=description
            )
            return await cls.get_instance(object_id)

        @classmethod
```

```
    async def delete_instance(cls, object_id):
        current = await cls.get_instance(object_id)
        if not current:
            return False
        await Project.filter(id=object_id).delete()
        return True

class TaskAdmin:
    @classmethod
    async def get_instances(cls):
        return await Task.all().order_by("-id").values()

    @classmethod
    async def get_instance(cls, object_id):
        return await Task.filter(id=object_id).first().values()

    @classmethod
    async def create_instance(cls, data):
        raise BadRequest("Create tasks through their project in the application.")

    @classmethod
    async def update_instance(cls, object_id, data):
        current = await cls.get_instance(object_id)
        if current is None:
            return None
        values = {**current, **task_fields(data, partial=True)}
        await Task.filter(id=object_id).update(
            **{key: values[key] for key in ("title", "status", "due_date")}
        )
        return await cls.get_instance(object_id)

    @classmethod
    async def delete_instance(cls, object_id):
        current = await cls.get_instance(object_id)
        if not current:
            return False
        await Task.filter(id=object_id).delete()
        return True

admin.register(
    ProjectAdmin,
    name="project",
    list_display=["id", "name", "owner_id"],
    search_fields=["name"],
    fields=["name", "description"],
    readonly_fields=["id", "owner_id"],
)
admin.register(
    TaskAdmin,

    name="task",
    list_display=["id", "title", "status", "due_date"],
    search_fields=["title"],
    list_filter=["status"],
    fields=["title", "status", "due_date"],
    readonly_fields=["id", "project_id"],
)
cms = CMS(app, auth=admin.auth)
cms.register(
    ContentType(
        "help_article",
        label="Help article",
        label_plural="Help articles",
        fields=[
```

```
                CMSField("title", required=True),
                CMSField("summary", type="textarea"),
                CMSField("body", type="richtext"),
            ],
            statuses=["draft", "published"],
            list_display=["title", "status"],
        )
    )
    cms.register(
        ContentType(
            "announcement",
            fields=[
                CMSField("title", required=True),
                CMSField("body", type="textarea"),
            ],
            statuses=["draft", "published"],
        )
    )
    app.backoffice, app.cms = admin, cms
```

Say: "These adapters query the same ORM models as the API and expose a deliberate staff field whitelist."

Type: Follow the action above in `backoffice.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 3: EDIT - replace the marked blocks - app.py

The application factory mounts only the features already introduced. Follow the action label: a marked-block edit keeps the surrounding factory intact.

Edit 1: find this exact block

Open `app.py`. Locate this block once. The unchanged surrounding lines identify its position; do not use line numbers from another revision.

```
from argon2 import PasswordHasher
from flaxon import Flaxon, Request
from flaxon.http import JSONResponse
from modules.auth.module import auth
from modules.projects.module import projects
from modules.tasks.module import tasks
```

Replace that block with

```
from argon2 import PasswordHasher
from flaxon import Flaxon, Request
from flaxon.http import JSONResponse
from backoffice import configure_backoffice
from modules.auth.module import auth
from modules.projects.module import projects
from modules.tasks.module import tasks
```

Leave the rest of the file unchanged.

Edit 2: find this exact block

Open `app.py`. Locate this block once. The unchanged surrounding lines identify its position; do not use line numbers from another revision.

```
    if app.is_management:
        return app

    app.use_teloce(
        project_root=ROOT,
        ui_dir="ui",
```

Replace that block with

```
    if app.is_management:
        return app

    configure_backoffice(
        app,
        admin_path or ADMIN_DATABASE_PATH,
        Path(admin_path or ADMIN_DATABASE_PATH).parent / "uploads",
    )
    app.use_teloce(
        project_root=ROOT,
        ui_dir="ui",
```

Leave the rest of the file unchanged.

Say: "The application factory mounts only the features already introduced. Follow the action label: a marked-block edit keeps the surrounding factory intact."

Type: Follow the action above in `app.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Run and check

```
python management.py setup-admin
python management.py runserver
# Open /admin/login with the staff account you just created.
```

Expected result

Staff sign-in works at /admin/login. The model lists show the same projects and tasks used by the customer API. A public customer account cannot access staff tools.

Common errors

Role labels alone do not grant every capability. Use project.view_project and task.view_task for readers; add change permissions only when needed. These staff adapters are global operations tools, not a tenant-scoped customer portal.

Short exercise

Create a read-only staff user and prove a direct edit request is denied as well as hiding its edit controls.

What to say and show

Say: "Create staff credentials and manage existing domain records with explicit permissions. The server remains responsible for persistence and authorization; the browser presents the result."

Show the expected result above, then point to the file responsible for it. Run the chapter check before the next lesson.

Save your checkpoint

```
git add .
git commit -m "Complete chapter 12: Staff Admin"
```

13 | CMS help and sample data

Goal

Publish sanitized help articles and show them through read-only public endpoints.

What you are building

CMS uses staff authentication. Help articles have title, summary, body and draft/published status. Public endpoints whitelist fields and expose only published items. Rich text is sanitized before the article component displays it with v-html; ordinary customer text remains escaped. seed is a project custom command exposed by flaxon_cli.py. It creates a sample project for the first registered customer and a published help article. Stop the server before seeding, then restart it so in-process CMS state reloads. Publishing permission remains distinct from drafting permission.

Build this chapter

Stop the server. Work in project_manager. Follow each CREATE, whole-file replacement, or marked-block edit below in order.

Step 1: CREATE - make parent folders, then create this file - flaxon_cli.py

This file owns the feature named by its module or component. Read the route/form flow before continuing.

Create the parent directories, then save this complete file.

```
"""Expose commands owned by feature modules to the Flaxon CLI.

Run these commands from the project directory. Add another module's
install_cli_commands(globals()) call here when you add its commands.
"""

from modules.welcome.module import welcome

welcome.install_cli_commands(globals())


from flaxon.cli.base import Command


def seed_command(args, console):
    import asyncio
    from seed import seed
    asyncio.run(seed())
    return 0


seed = Command("seed", seed_command, help_text="Create sample projects and published help
    content")
```

Say: "This file owns the feature named by its module or component. Read the route/form flow before continuing."

Type: Follow the action above in `flaxon_cli.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 2: CREATE - make parent folders, then create this file - modules/content/__init__.py

This file owns the feature named by its module or component. Read the route/form flow before continuing.

Create the parent directories, then save this complete file.

```
"""Module-owned APIs and interface files."""
```

Say: "This file owns the feature named by its module or component. Read the route/form flow before continuing."

Type: Follow the action above in `modules/content/__init__.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 3: CREATE - make parent folders, then create this file - modules/content/module.py

Create the public content API. Return only published CMS help articles. Staff draft and publishing operations remain in Admin.

Create the parent directories, then save this complete file.

```
"""Public read-only content endpoints; never expose the authenticated CMS API to readers."""

from pathlib import Path
from flaxon.modules import FlaxonModule
from flaxon.exceptions import NotFound

content = FlaxonModule(
    "content",
    ui_dir=Path(__file__).parent / "ui",
    ui_routes={"Help.js": "/help", "Article/[slug].js": "/help/:slug"},
)


async def published_articles(request):
    cms = request.app.cms
    await cms._load_database()
    articles = cms.content_types["help_article"]
    return [
        item for item in articles.items.values() if item.get("status") == "published"
    ]


@content.get("/articles")
async def list_articles(request):
    articles = await published_articles(request)
    return {
```

```
        "data": [
            {key: item.get(key) for key in ("id", "title", "slug", "summary")}
            for item in articles
        ]
    }


@content.get("/articles/<slug>")

async def get_article(request, slug):
    for article in await published_articles(request):
        if article["slug"] == slug:
            return {
                "data": {
                    key: article.get(key)
                    for key in ("id", "title", "slug", "summary", "body")
                }
            }
    raise NotFound("Article not found.")
```

Say: "Create the public content API. Return only published CMS help articles. Staff draft and publishing operations remain in Admin."

Type: Follow the action above in `modules/content/module.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 4: CREATE - make parent folders, then create this file - modules/content/ui/pages/Help.html

This file owns the feature named by its module or component. Read the route/form flow before continuing.

Create the parent directories, then save this complete file.

```
<template>
  <section>
    <h1>Help centre</h1>
    <p v-if="loading" role="status">Loading articles...</p>
    <p v-if="message" role="alert">{{ message }}</p>
    <p v-if="!loading && articles.length === 0">No published articles yet.</p>
    <div class="cards">
      <a
        v-for="article in articles"
        :key="article.id"
        :href="'/help/' + article.slug"
        data-teloce-link
        class="card"
        ><h2>{{ article.title }}</h2>
        <p>{{ article.summary }}</p></a
      >
    </div>
  </section>
</template>
<script lang="ts">
import { api } from "../../../../ui/api.ts";
export default {
  data() {
    return { articles: [], loading: true, message: "" };
  },
```

```
  async mounted() {
    try {
      this.articles = await api("/api/content/articles");
    } catch (error) {
      this.message = error.message;
    } finally {
      this.loading = false;
    }
  },

};
</script>
<style scoped>
* {
  box-sizing: border-box;
}
a {
  color: #1d4ed8;
}
h1 {
  font-size: clamp(1.8rem, 5vw, 3rem);
  line-height: 1.15;
}
h2 {
  font-size: 1.2rem;
}
p {
  line-height: 1.6;
}
[role="alert"] {
  color: #a21c2b;
  padding: 0.8rem;
  border-left: 4px solid;
}
:focus-visible {
  outline: 3px solid #f59e0b;
  outline-offset: 3px;
}
.card {
  display: block;
  background: white;
  border: 1px solid #d8e0ed;
  border-radius: 12px;
  padding: 1.2rem;

  margin: 1rem 0;
  color: inherit;
  text-decoration: none;
}
.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(min(100%, 240px), 1fr));
  gap: 1rem;
}
</style>
```

Say: "This file owns the feature named by its module or component. Read the route/form flow before continuing."

Type: Follow the action above in `modules/content/ui/pages/Help.html`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 5: CREATE - make parent folders, then create this file - modules/content/ui/pages/Article/[slug].html

This file owns the feature named by its module or component. Read the route/form flow before continuing.

Create the parent directories, then save this complete file.

```
<template>
  <article>
    <a href="/help" data-teloce-link><- Help centre</a>
    <p v-if="loading" role="status">Loading article...</p>
    <p v-if="message" role="alert">{{ message }}</p>
    <div v-if="article">
      <h1>{{ article.title }}</h1>
      <p>{{ article.summary }}</p>
      <div class="article-body" v-html="article.body"></div>
    </div>
  </article>
</template>
<script lang="ts">
import { api } from "../../../../../ui/api.ts";
export default {
  props: ["slug"],
  data() {
    return { article: null, loading: true, message: "" };
  },
  async mounted() {
    try {
      this.article = await api(
        "/api/content/articles/" + encodeURIComponent(this.slug),
      );
    } catch (error) {
      this.message = error.message;
    } finally {
      this.loading = false;
    }
  },
};
</script>
<style scoped>
* {

  box-sizing: border-box;
}
a {
  color: #1d4ed8;
}
h1 {
  font-size: clamp(1.8rem, 5vw, 3rem);
  line-height: 1.15;
}
h2 {
  font-size: 1.2rem;
}
p {
  line-height: 1.6;
}
[role="alert"] {
  color: #a21c2b;
```

```
  padding: 0.8rem;
  border-left: 4px solid;
}
:focus-visible {
  outline: 3px solid #f59e0b;
  outline-offset: 3px;
}
.article-body {
  line-height: 1.6;
  overflow-wrap: anywhere;
}
</style>
```

Say: "This file owns the feature named by its module or component. Read the route/form flow before continuing."

Type: Follow the action above in `modules/content/ui/pages/Article/[slug].html`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 6: CREATE - make parent folders, then create this file - seed.py

Populate sample data explicitly after a customer exists; do not seed on every server start.

Create the parent directories, then save this complete file.

```
"""Idempotent sample content. Register your own user before running this command."""

from app import create_app
from models import User, Project, Task


async def seed():
    app = create_app()
    async with app.db:
        user = await User.all().order_by("id").first()
        if user is None:
            raise ValueError("Register an account before seeding sample projects.")
        if not await Project.filter(owner_id=user.id).exists():
            project = await Project.create(
                owner_id=user.id,
                name="Launch my portfolio",
                description="A small project to practise planning.",
            )
            for title, status in [
                ("Choose a design", "done"),
                ("Build the homepage", "doing"),
                ("Deploy the website", "todo"),
            ]:
                await Task.create(project_id=project.id, title=title, status=status)
    articles = app.cms.content_types["help_article"]
    if not articles.items:
        articles.create(
            {
                "title": "Getting started",
                "summary": "Create a project, add tasks, and track progress.",
                "body": "<p>Open Projects, create a project, and add your first task. Move
    tasks from To do to Doing to Done.</p>",
                "status": "published",
            }
```

```
        )
        app.cms._save(articles)
    print("Sample projects, tasks, and help content are ready.")
```

Say: "Populate sample data explicitly after a customer exists; do not seed on every server start."

Type: Follow the action above in `seed.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 7: EDIT - replace the marked blocks - ui/app.html

This is the SPA shell. Its internal anchors and router view cooperate with each module's page mappings.

Edit 1: find this exact block

Open `ui/app.html`. Locate this block once. The unchanged surrounding lines identify its position; do not use line numbers from another revision.

```
      <a href="/" data-teloce-link class="brand">Project Manager</a>
      <nav aria-label="Main navigation">
        <a href="/projects" data-teloce-link>Projects</a>
        <a href="/login" data-teloce-link>Account</a>
      </nav>
    </header>
```

Replace that block with

```
      <a href="/" data-teloce-link class="brand">Project Manager</a>
      <nav aria-label="Main navigation">
        <a href="/projects" data-teloce-link>Projects</a>
        <a href="/help" data-teloce-link>Help</a>
        <a href="/login" data-teloce-link>Account</a>
      </nav>
    </header>
```

Leave the rest of the file unchanged.

Say: "This is the SPA shell. Its internal anchors and router view cooperate with each module's page mappings."

Type: Follow the action above in `ui/app.html`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 8: EDIT - replace the marked blocks - app.py

The application factory mounts only the features already introduced. Follow the action label: a marked-block edit keeps the surrounding factory intact.

Edit 1: find this exact block

Open `app.py`. Locate this block once. The unchanged surrounding lines identify its position; do not use line numbers from another revision.

```
from modules.auth.module import auth
from modules.projects.module import projects
from modules.tasks.module import tasks
```

```
from modules.welcome.module import welcome
from settings import (
    ROOT,
```

Replace that block with

```
from modules.auth.module import auth
from modules.projects.module import projects
from modules.tasks.module import tasks
from modules.content.module import content
from modules.welcome.module import welcome
from settings import (
    ROOT,
```

Leave the rest of the file unchanged.

Edit 2: find this exact block

Open app.py. Locate this block once. The unchanged surrounding lines identify its position; do not use line numbers from another revision.

```
        (auth, "/api/auth"),
        (projects, "/api/projects"),
        (tasks, "/api/tasks"),
        (welcome, "/api/welcome"),
    ]:
        app.mount_module(module, prefix=prefix)
```

Replace that block with

```
        (auth, "/api/auth"),
        (projects, "/api/projects"),
        (tasks, "/api/tasks"),
        (content, "/api/content"),
        (welcome, "/api/welcome"),
    ]:
        app.mount_module(module, prefix=prefix)
```

Leave the rest of the file unchanged.

Edit 3: find this exact block

Open app.py. Locate this block once. The unchanged surrounding lines identify its position; do not use line numbers from another revision.

```
    @app.get("/login")
    @app.get("/projects")
    @app.get("/projects/<int:project_id>")
    async def spa(request: Request, project_id=None, slug=None):
        return await request.compile("app.html", {})
```

Replace that block with

```
    @app.get("/login")
    @app.get("/projects")
    @app.get("/projects/<int:project_id>")
    @app.get("/help")
    @app.get("/help/<slug>")
    async def spa(request: Request, project_id=None, slug=None):
        return await request.compile("app.html", {})
```

Leave the rest of the file unchanged.

Say: "The application factory mounts only the features already introduced. Follow the action label: a marked-block edit keeps the surrounding factory intact."

Type: Follow the action above in `app.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Run and check

```
python management.py seed
# Stop/restart the running server after seeding:
python management.py runserver
# Open /help and /admin/cms/.
```

Expected result

Run seed after registering a customer. /help lists the sample published article and the article page survives a refresh. Drafts remain absent from public responses.

Common errors

Do not call authenticated CMS editing endpoints from public reader pages. Do not display arbitrary HTML using v-html. A draft editor also needs publishing permission to modify live content.

Short exercise

Create a draft in staff CMS, confirm it is hidden publicly, then publish it with a permitted staff account.

What to say and show

Say: "Publish sanitized help articles and show them through read-only public endpoints. The server remains responsible for persistence and authorization; the browser presents the result."

Show the expected result above, then point to the file responsible for it. Run the chapter check before the next lesson.

Save your checkpoint

```
git add .
git commit -m "Complete chapter 13: CMS help and sample data"
```

14 | Full-stack verification

Goal

Exercise customer, staff and CMS workflows in a real browser.

What you are building

API tests cannot prove form listeners, history navigation or mobile layout. The browser script starts a disposable server, seeds a temporary staff account and tests desktop and 390-pixel workflows. It checks registration, login, projects, tasks, progress, help, navigation, refresh and logout. Run it in development and production mode to test MinifyJS output. Use labels, status messages and alerts; also rehearse with the keyboard.

Build this chapter

Stop the server. Work in project_manager. Follow each CREATE, whole-file replacement, or marked-block edit below in order.

Step 1: CREATE - make parent folders, then create this file - tests/test_backoffice.py

This file owns the feature named by its module or component. Read the route/form flow before continuing.

Create the parent directories, then save this complete file.

```
import asyncio
from flaxon import Flaxon
from flaxon.admin.cms import CMS, CMSField, ContentType
from flaxon.testing import TestClient


def staff(application, permissions):
    admin = application.backoffice
    admin.auth.add_user(
        {
            "username": "editor",
            "password": "EditorLearning123!",
            "roles": [],
            "permissions": permissions,
        }
    )
    token = asyncio.run(admin.auth.login("editor", "EditorLearning123!"))
    return {"cookie": f"session_id={token}", "x-csrf-token": admin.csrf_token()}


def test_public_content_hides_drafts_and_private_fields(application, client):
    articles = application.cms.content_types["help_article"]
    articles.create({"title": "Hidden draft", "status": "draft"})
    item = articles.create(
        {
            "title": "Published guide",
            "body": "<p>Safe</p><script>bad()</script>",
            "status": "published",
        }
    )
```

```
    response = client.get("/api/content/articles")
    assert [row["title"] for row in response.json()["data"]] == ["Published guide"]
    detail = client.get("/api/content/articles/" + item["slug"]).json()["data"]
    assert "<script>" not in detail["body"]

    assert "created_at" not in detail
    assert client.get("/api/content/articles/hidden-draft").status_code == 404


def test_application_accounts_do_not_have_staff_access(client, account):
    assert account.call("/admin/cms/api/help_article/items").status_code == 401


def test_editor_can_write_drafts_but_cannot_publish(application, client):
    headers = staff(
        application,
        [
            "help_article.view_help_article",
            "help_article.add_help_article",
            "help_article.change_help_article",
        ],
    )
    path = "/admin/cms/api/help_article/items"
    assert (
        client.post(path, json_data={"title": "Draft"}, headers=headers).status_code
        == 201
    )
    assert (
        client.post(
            path,
            json_data={"title": "Forbidden", "status": "published"},
            headers=headers,
        ).status_code
        == 403
    )
    assert (
        client.post(
            "/admin/cms/api/import/help_article",
            json_data=[{"title": "Forbidden", "status": "published"}],

            headers=headers,
        ).status_code
        == 403
    )


def test_publisher_requires_csrf(application, client):
    headers = staff(
        application, ["help_article.add_help_article", "cms.publish_content"]
    )
    path = "/admin/cms/api/help_article/items"
    assert (
        client.post(
            path,
            json_data={"title": "Published", "status": "published"},
            headers=headers,
        ).status_code
        == 201
    )
    assert (
        client.post(
            path,
            json_data={"title": "Missing token"},
            headers={"cookie": headers["cookie"]},
```

```
        ).status_code
        == 400
    )


def test_standalone_cms_is_closed_by_default():
    app = Flaxon("cms-test", debug=True)
    cms = CMS(app)
    cms.register(ContentType("article", fields=[CMSField("title")]))
    assert (

        TestClient(app)
        .post("/admin/cms/api/article/items", json_data={"title": "Forbidden"})
        .status_code
        == 403
    )


def test_database_and_cms_survive_new_application_instance(
    application, client, account, tmp_path
):
    from app import create_app

    account.call("/api/projects/", "post", {"name": "Persistent"})
    articles = application.cms.content_types["help_article"]
    articles.create({"title": "Persistent article", "status": "published"})
    application.cms._save(articles)
    new_app = create_app(
        tmp_path / "app.sqlite3", tmp_path / "admin.sqlite3", debug=True
    )

    async def read_project():
        from models import Project

        async with new_app.db:
            return (await Project.all().first()).name

    assert asyncio.run(read_project()) == "Persistent"
    assert len(new_app.cms.content_types["help_article"].items) == 1


def test_staff_model_adapter_reads_domain_records_and_denies_ungranted_changes(
    application, client, account
):
    account.call("/api/projects/", "post", {"name": "Domain record"})

    headers = staff(application, ["admin.view_dashboard", "project.view_project"])
    response = client.get("/admin/project", headers=headers)
    assert response.status_code == 200
    assert "Domain record" in response.text
    assert client.get("/admin/project/1/edit", headers=headers).status_code == 403
    assert client.post("/admin/project/1/delete", headers=headers).status_code == 403
```

Say: "This file owns the feature named by its module or component. Read the route/form flow before continuing."

Type: Follow the action above in `tests/test_backoffice.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 2: CREATE - make parent folders, then create this file - tests/test_cms_security.py

This file owns the feature named by its module or component. Read the route/form flow before continuing.

Create the parent directories, then save this complete file.

```
import asyncio

import pytest
from flaxon import Flaxon
from flaxon.admin import AdminDashboard
from flaxon.admin.cms import CMS, CMSField, ContentType
from flaxon.testing import TestClient


def setup_editor():
    app = Flaxon("cms-security", debug=True)
    admin = AdminDashboard(
        app,
        strict_permissions=True,
        users=[
            {
                "username": "editor",
                "password": "Editor123!",
                "roles": [],
                "permissions": [
                    "article.view_article",
                    "article.add_article",
                    "article.change_article",
                    "cms.restore_revision",
                ],
            }
        ],
    )
    cms = CMS(app, auth=admin.auth)
    content = cms.register(
        ContentType("article", fields=[CMSField("title", required=True)])
    )
    token = asyncio.run(admin.auth.login("editor", "Editor123!"))
    headers = {"cookie": f"session_id={token}", "x-csrf-token": admin.csrf_token()}

    return TestClient(app), cms, content, headers


def test_standalone_cms_denies_anonymous_requests_by_default():
    app = Flaxon("closed-cms")
    cms = CMS(app)
    cms.register(ContentType("article", fields=[CMSField("title")]))
    client = TestClient(app)
    assert client.get("/admin/cms/api/article/items").status_code == 403
    assert (
        client.post(
            "/admin/cms/api/article/items", json_data={"title": "Anonymous"}
        ).status_code
        == 403
    )


@pytest.mark.parametrize("status", ["approved", "scheduled", "published"])
@pytest.mark.parametrize("method", ["create", "update", "import"])
def test_editor_cannot_publish_through_generic_write_paths(status, method):
    client, cms, content, headers = setup_editor()
```

```
    data = {"title": "Blocked", "status": status}
    if method == "create":
        response = client.post(
            "/admin/cms/api/article/items", json_data=data, headers=headers
        )
    elif method == "update":
        item = content.create({"title": "Draft"})
        response = client.put(
            f"/admin/cms/api/article/items/{item['id']}",
            json_data=data,
            headers=headers,
        )
        assert item["status"] == "draft"

    else:
        response = client.post(
            "/admin/cms/api/import/article", json_data=[data], headers=headers
        )
    assert response.status_code == 403
    assert not any(item["status"] == status for item in content.items.values())


def test_editor_cannot_restore_published_revision_or_edit_published_content():
    client, cms, content, headers = setup_editor()
    item = content.create({"title": "Published", "status": "published"})
    assert (
        client.put(
            f"/admin/cms/api/article/items/{item['id']}",
            json_data={"title": "Changed"},
            headers=headers,
        ).status_code
        == 403
    )
    content.update(item["id"], {"status": "draft"})
    assert (
        client.post(
            f"/admin/cms/api/article/items/{item['id']}/restore/0", headers=headers
        ).status_code
        == 403
    )
    assert item["status"] == "draft"


def test_draft_editing_and_publisher_creation_work_with_csrf():
    client, cms, content, headers = setup_editor()
    assert (
        client.post(
            "/admin/cms/api/article/items",

            json_data={"title": "Draft"},
            headers=headers,
        ).status_code
        == 201
    )
    cms.auth.users["editor"]["permissions"].append("cms.publish_content")
    assert (
        client.post(
            "/admin/cms/api/article/items",
            json_data={"title": "Published", "status": "published"},
            headers=headers,
        ).status_code
        == 201
    )
    assert (
```

```
        client.post(
            "/admin/cms/api/article/items",
            json_data={"title": "No CSRF"},
            headers={"cookie": headers["cookie"]},
        ).status_code
        == 400
    )


def test_import_authorization_is_checked_before_any_row_is_created():
    client, cms, content, headers = setup_editor()
    response = client.post(
        "/admin/cms/api/import/article",
        json_data=[
            {"title": "Draft"},
            {"title": "Forbidden", "status": "published"},
        ],
        headers=headers,
    )

    assert response.status_code == 403
    assert content.items == {}


@pytest.mark.parametrize("action", ["publish", "unpublish", "custom"])
def test_actions_require_publisher_permission(action):
    client, cms, content, headers = setup_editor()
    item = content.create({"title": "Draft"})
    content.register_action("custom", "Custom", lambda content_type, ids: None)
    response = client.post(
        f"/admin/cms/api/article/actions/{action}",
        json_data={"ids": [item["id"]]},
        headers=headers,
    )
    assert response.status_code == 403
```

Say: "This file owns the feature named by its module or component. Read the route/form flow before continuing."

Type: Follow the action above in `tests/test_cms_security.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 3: CREATE - make parent folders, then create this file - scripts/browser_smoke.py

This file owns the feature named by its module or component. Read the route/form flow before continuing.

Create the parent directories, then save this complete file.

```
"""Run desktop/mobile workflows against a fresh temporary database. No real user data."""

import os
import subprocess
import sys
import secrets
import tempfile
import time
from pathlib import Path
```

```
import httpx
from playwright.sync_api import sync_playwright

ROOT = Path(__file__).resolve().parents[1]
BASE = "http://127.0.0.1:8123"


def main():
    with tempfile.TemporaryDirectory() as directory:
        production = "--production" in sys.argv
        env = {
            **os.environ,
            "DATA_DIR": directory,
            "FLAXON_ALLOWED_HOSTS": "127.0.0.1",
            "FLAXON_DEBUG": "0" if production else "1",
            "PUBLIC_ORIGIN": BASE,
            "FLAXON_SECRET_KEY": secrets.token_urlsafe(48),
            "COURSE_SMOKE_STAFF_PASSWORD": "SmokeAa1!" + secrets.token_urlsafe(24),
        }
        subprocess.run(
            [sys.executable, "management.py", "migrate"], cwd=ROOT, env=env, check=True
        )
        # Bootstrap a staff account in the disposable Admin store before startup.
        bootstrap = """
import os

from flaxon.admin.services import AdminAuth, AdminStore
from settings import ADMIN_DATABASE_PATH
store = AdminStore(str(ADMIN_DATABASE_PATH))
auth = AdminAuth(users=[], store=store, strict_permissions=True)
record = auth.add_user({'username': 'course-admin', 'password':
    os.environ['COURSE_SMOKE_STAFF_PASSWORD'], 'roles': ['administrator']})
store.set('users', 'course-admin', record)
"""
        subprocess.run([sys.executable, "-c", bootstrap], cwd=ROOT, env=env, check=True)
        log = open(Path(directory) / "server.log", "w")
        server = subprocess.Popen(
            [sys.executable, "-m", "uvicorn", "app:app", "--port", "8123"],
            cwd=ROOT,
            env=env,
            stdout=log,
            stderr=log,
        )
        try:
            for _ in range(150):
                try:
                    if httpx.get(BASE + "/", trust_env=False).status_code == 200:
                        break
                except httpx.ConnectError:
                    time.sleep(0.1)
            else:
                raise RuntimeError("Server failed to start")
            with sync_playwright() as playwright:
                browser = playwright.chromium.launch(headless=True)
                staff_context = browser.new_context()
                staff_page = staff_context.new_page()
                staff_page.goto(BASE + "/admin/login")
                staff_page.get_by_label("Username", exact=True).fill("course-admin")
                staff_page.get_by_label("Password", exact=True).fill(
                    env["COURSE_SMOKE_STAFF_PASSWORD"]

                )
                staff_page.get_by_role("button", name="Sign in", exact=True).click()
                staff_page.wait_for_url("**/admin/")
```

```
            assert (
                staff_page.evaluate(
                    "async () => (await fetch('/admin/project')).status"
                )
                == 200
            )
            staff_page.goto(BASE + "/admin/cms/")
            csrf = staff_page.locator('meta[name="csrf-token"]').get_attribute(
                "content"
            )
            response = staff_page.evaluate(
                """async (csrf) => {
                const response = await fetch('/admin/cms/api/help_article/items', {
                    method: 'POST', headers: {'Content-Type': 'application/json',
'X-CSRF-Token': csrf},
                    body: JSON.stringify({title: 'Browser help guide', summary: 'Published
 by staff', body: '<p>Safe help content.</p>', status: 'published'})
                });
                return {status: response.status, body: await response.text()};
            }""",
                csrf,
            )
            assert response["status"] == 201, response["body"]
            staff_context.close()
            print("Staff Admin login and CMS publication passed")
            for width in [1440, 390]:
                context = browser.new_context(
                    viewport={"width": width, "height": 900}
                )
                page = context.new_page()
                errors = []

                page.on("pageerror", lambda error: errors.append(str(error)))
                page.goto(BASE + "/login")
                register_button = page.get_by_role("button", name="Register instead")
                assert register_button.evaluate(
                    "element => getComputedStyle(element).backgroundColor"
                ) == "rgb(228, 234, 245)", "Login scoped secondary button style missing"
                assert page.evaluate(
                    "getComputedStyle(document.body).margin === '0px'"
                ), "Shell document reset missing"
                register_button.click()
                page.get_by_label("Name", exact=True).fill("Course Learner")
                page.get_by_label("Email", exact=True).fill(
                    f"learner{width}@example.test"
                )
                page.get_by_label("Password", exact=True).fill("SmokeAa1!" +
secrets.token_urlsafe(24))
                page.get_by_role(
                    "button", name="Create account", exact=True
                ).click()
                try:
                    page.wait_for_url("**/projects", timeout=8000)
                except Exception:
                    print("Browser errors:", errors)
                    print("URL:", page.url)
                    print("Page:", page.locator("body").inner_text())
                    raise
                page.get_by_label("Name", exact=True).fill("Browser project")
                page.get_by_label("Description", exact=True).fill(
                    "Verified in the browser"
                )
                page.get_by_role(
                    "button", name="Create project", exact=True
                ).click()
```

```
                page.get_by_role("link", name="Browser project").click()

                page.get_by_role(
                    "heading", name="Browser project", exact=True
                ).wait_for()
                add_task_button = page.get_by_role("button", name="Add task", exact=True)
                assert add_task_button.evaluate(
                    "element => getComputedStyle(element).backgroundColor"
                ) == "rgb(37, 99, 235)", "TaskForm must own its scoped button style"
                assert page.evaluate("""() => {
                    const probe = document.createElement('button');
                    probe.textContent = 'Unscoped probe';
                    document.body.appendChild(probe);
                    const colour = getComputedStyle(probe).backgroundColor;
                    probe.remove();
                    return colour !== 'rgb(37, 99, 235)';
                }"""), "Component button styles leaked outside their scope"
                assert not any('/assets/app.css' in entry['name'] for entry in
                    page.evaluate("performance.getEntriesByType('resource').map(entry =>
({name: entry.name}))")
                ), "The SPA must not load a shared app.css stylesheet"
                page.get_by_label("Task title").fill("Record the lesson")
                page.get_by_role("button", name="Add task", exact=True).click()
                page.get_by_text("Record the lesson", exact=True).wait_for()
                page.get_by_label("Task status", exact=True).select_option("done")
                page.get_by_text("100% complete", exact=True).wait_for()
                screenshot_dir = os.getenv("COURSE_SCREENSHOT_DIR")
                if screenshot_dir:
                    Path(screenshot_dir).mkdir(parents=True, exist_ok=True)
                    page.screenshot(path=str(Path(screenshot_dir) /
f"project-{width}.png"), full_page=True)
                page.get_by_label("Filter tasks").select_option("todo")
                page.get_by_text(
                    "No tasks match this filter.", exact=True
                ).wait_for()
                page.get_by_label("Filter tasks").select_option("all")

                page.evaluate("window.courseNavigationMarker = 42")
                page.get_by_role("link", name="Help", exact=True).click()
                page.get_by_role("heading", name="Help centre").wait_for()
                assert (
                    page.evaluate("window.courseNavigationMarker") == 42
                ), "SPA link reloaded the document"
                page.get_by_role("link", name="Browser help guide").click()
                page.get_by_role(
                    "heading", name="Browser help guide", exact=True
                ).wait_for()
                page.get_by_text("Safe help content.", exact=True).wait_for()
                page.go_back()
                page.get_by_role("heading", name="Help centre").wait_for()
                page.go_back()
                page.get_by_role(
                    "heading", name="Browser project", exact=True
                ).wait_for()
                page.reload()
                page.get_by_text("100% complete", exact=True).wait_for()
                assert page.evaluate(
                    "document.documentElement.scrollWidth <= innerWidth"
                ), "Mobile overflow"
                page.get_by_role("link", name="Account", exact=True).click()
                page.get_by_role("button", name="Sign out").click()
                page.get_by_text("Signed out.", exact=True).wait_for()
                assert page.request.get(BASE + "/api/projects/").status == 401
                assert not errors, errors
```

```
                    context.close()
                    print(
                        f"{'Production MinifyJS' if production else 'Development'} browser
    workflow passed at {width}px"
                    )
                browser.close()
        finally:

            server.terminate()
            server.wait(timeout=10)
            log.close()


if __name__ == "__main__":
    main()
```

Say: "This file owns the feature named by its module or component. Read the route/form flow before continuing."

Type: Follow the action above in `scripts/browser_smoke.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Run and check

```
python -m playwright install chromium
python -m pytest -q
python scripts/browser_smoke.py
python scripts/browser_smoke.py --production
```

Expected result

All application tests and both browser modes pass. Test data stays outside your learner database. No browser console errors occur in the verified workflow.

Common errors

Install Chromium before Playwright. Stop other servers using port 8123. Loopback production smoke verifies assets and behavior, not cloud infrastructure or TLS.

Short exercise

Complete the workflow using only the keyboard and check one layout change at both desktop and mobile sizes.

What to say and show

Say: "Exercise customer, staff and CMS workflows in a real browser. The server remains responsible for persistence and authorization; the browser presents the result."

Show the expected result above, then point to the file responsible for it. Run the chapter check before the next lesson.

Save your checkpoint

```
git add .
git commit -m "Complete chapter 14: Full-stack verification"
```

15 | Production build and Render

Goal

Build optimized assets and deploy one persistent application instance.

What you are building

The Blueprint installs pinned dependencies and builds Teloce with MinifyJS. The start command applies committed Python migrations, then starts one Uvicorn worker. SQLite and staff/CMS data live under /var/data on a paid persistent disk. Render's build and pre-deploy processes cannot access that disk, which is why this design migrates at startup. Set PUBLIC_ORIGIN to your exact HTTPS service URL, DEBUG to false and a persistent generated secret. This course deliberately uses one instance: scaling needs a shared database, sessions, media and CMS coordination strategy.

Build this chapter

Stop the server. Work in project_manager. Follow each CREATE, whole-file replacement, or marked-block edit below in order.

Step 1: CREATE - make parent folders, then create this file - .env.example

This file owns the feature named by its module or component. Read the route/form flow before continuing.

Create the parent directories, then save this complete file.

```
FLAXON_DEBUG=1
PUBLIC_ORIGIN=http://127.0.0.1:8000
# Production: FLAXON_DEBUG=0, PUBLIC_ORIGIN=https://your-app.onrender.com
# Render persistent disk mount: DATA_DIR=/var/data
# Generate with: python -c "import secrets; print(secrets.token_urlsafe(48))"
FLAXON_SECRET_KEY=
```

Say: "This file owns the feature named by its module or component. Read the route/form flow before continuing."

Type: Follow the action above in `.env.example`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 2: CREATE - make parent folders, then create this file - scripts/build_ui.py

This file owns the feature named by its module or component. Read the route/form flow before continuing.

Create the parent directories, then save this complete file.

```
"""Build production Teloce assets without touching the live database or Render disk."""

import os
import secrets
```

```
import sys
import tempfile
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
with tempfile.TemporaryDirectory() as build_data:
    os.environ["DATA_DIR"] = build_data
    os.environ["FLAXON_DEBUG"] = "0"
    os.environ.setdefault("FLAXON_SECRET_KEY", secrets.token_urlsafe(48))
    from app import app

    result = app.teloce.build()
    print(f"Built {result.get('compiled', 0)} Teloce components with MinifyJS.")
```

Say: "This file owns the feature named by its module or component. Read the route/form flow before continuing."

Type: Follow the action above in `scripts/build_ui.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 3: CREATE - make parent folders, then create this file - render.yaml

This is the complete single-instance deployment definition. Supply the service-specific HTTPS origin in Render.

Create the parent directories, then save this complete file.

```
services:
  - type: web
    name: flaxon-project-manager
    runtime: python
    plan: starter
    buildCommand: python scripts/verify_vendor.py && pip install -r requirements.txt && python
     scripts/build_ui.py
    startCommand: python management.py migrate && python -m uvicorn app:app --host 0.0.0.0
    --port $PORT --workers 1
    healthCheckPath: /health/course
    disk:
      name: project-manager-data
      mountPath: /var/data
      sizeGB: 1
    envVars:
      - key: PYTHON_VERSION
        value: "3.12.14"
      - key: FLAXON_DEBUG
        value: "0"
      - key: FLAXON_SECRET_KEY
        generateValue: true
      - key: DATA_DIR
        value: /var/data
      - key: PUBLIC_ORIGIN
        sync: false
```

Say: "This is the complete single-instance deployment definition. Supply the service-specific HTTPS origin in Render."

Type: Follow the action above in `render.yaml`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Run and check

```
python scripts/build_ui.py
python -m pip check
```

Release-only edit: Render installation

If you chose the published Flaxon 3.0.0 path, use `requirements-release.txt` as your project's `requirements.txt`. In `render.yaml`, find:

```
buildCommand: python scripts/verify_vendor.py && pip install -r requirements.txt && python
 scripts/build_ui.py
```

Replace only that line with:

```
buildCommand: pip install -r requirements.txt && python scripts/build_ui.py
```

Leave the rest of the Blueprint unchanged. The preview path keeps wheel verification and includes `vendor/`. The release path installs the pinned published packages and does not need preview wheels. This release branch must pass the clean course rehearsal after publication before you record it.

Deploy the completed project

1. Stop the local server. Run all tests and both browser checks from chapter 14. Run the production build above.

2. Commit your Python migrations, application files, vendor wheels and requirements. Keep .env, data, cookie jars and local virtual environments out of Git. Push the project to your GitHub repository.

3. In Render, choose New > Blueprint, connect your repository and select the branch containing render.yaml. Review the paid service and persistent disk before deploying.

4. Supply PUBLIC_ORIGIN as the exact https://SERVICE.onrender.com address (or your configured HTTPS domain). Keep FLAXON_DEBUG=0, DATA_DIR=/var/data and the generated persistent secret. Python is pinned by the Blueprint.

5. Deploy and inspect logs: migrations must complete before Uvicorn listens. Do not move SQLite migrations to a pre-deploy command; that process cannot access the attached disk.

6. In the service Shell run python management.py setup-admin and enter staff credentials interactively. Never commit a staff password.

7. Visit /health/course, then register a customer, create a project and task, sign into staff Admin and publish a help article. Refresh /projects/ID and /help/SLUG. Sign out and check protected API requests are rejected.

8. Restart the service and confirm both the project and help article survive. Inspect browser cookies over HTTPS for Secure and HttpOnly. Back up data and uploads before upgrades.

Official deployment references, checked October 2026: Render Blueprints, persistent disks, deployment lifecycle, and Python versions. This book supplies deployable configuration; it does not claim a deployment has been made in your account.

Expected result

After deployment, /health/course returns ok. Create a project/task, publish an article, refresh a nested route, sign out and restart the service. Customer and staff data survive; cookies use Secure and HttpOnly.

Common errors

Free Render services cannot attach this persistent disk. Do not generate new migration files during deployment: commit them beforehand. An incorrect PUBLIC_ORIGIN causes CSRF failures. An ephemeral database loses records on restart. Production deployment must be verified in your account; local smoke is not proof of a live deployment.

Short exercise

Back up both SQLite files safely, restore them into a test environment and verify a project plus one published help article.

What to say and show

Say: "Build optimized assets and deploy one persistent application instance. The server remains responsible for persistence and authorization; the browser presents the result."

Show the expected result above, then point to the file responsible for it. Run the chapter check before the next lesson.

Save your checkpoint

```
git add .
git commit -m "Complete chapter 15: Production build and Render"
```

16 | PWA: installation, offline fallback and updates

Goal

Turn the completed SPA into an optional installable application without caching private data.

What you are building

Enable Teloce's first-party PWA plugin through Flaxon. Mount generated icons, add a scoped status component and rebuild production assets. The browser receives a manifest and a same-origin service worker. It caches emitted JS/CSS and a generic offline page, not API or Admin responses. Offline means the app can explain a lost connection; it does not save tasks offline. Registration is disabled in development, so check a production build over localhost or HTTPS. This is an optional final lesson, after the existing deployment chapter.

Build this chapter

Stop the server. Work in project_manager. Follow each CREATE, whole-file replacement, or marked-block edit below in order.

Step 1: CREATE - make parent folders, then create this file - scripts/create_pwa_icons.py

Create the icon generator. It writes two PNG files using the standard library. Run it before the development server; production builds invoke it automatically.

Create the parent directories, then save this complete file.

```
"""Generate portable course icons using only Python's standard library."""
from pathlib import Path
import struct
import zlib

ROOT = Path(__file__).resolve().parents[1]


def chunk(kind, data):
    checksum = zlib.crc32(kind + data) & 0xFFFFFFFF
    return struct.pack(">I", len(data)) + kind + data + struct.pack(">I", checksum)


def icon(size):
    rows = bytearray()
    for y in range(size):
        rows.append(0)  # PNG filter: none.
        for x in range(size):
            px, py = x / size, y / size
            color = (23, 36, 58)
            # A folder and a check mark, with generous maskable-icon padding.
            if 0.20 <= px <= 0.80 and 0.34 <= py <= 0.72:
                color = (219, 234, 254)
            if 0.20 <= px <= 0.45 and 0.27 <= py <= 0.35:
                color = (219, 234, 254)
            if 0.37 <= px <= 0.51 and abs(py - (px + 0.13)) < 0.035:
```

```
                color = (20, 100, 170)
            if 0.49 <= px <= 0.68 and abs(py - (-px + 1.13)) < 0.035:
                color = (20, 100, 170)
            rows.extend(color)
    header = struct.pack(">IIBBBBB", size, size, 8, 2, 0, 0, 0)
    return b"\x89PNG\r\n\x1a\n" + chunk(b"IHDR", header) + chunk(b"IDAT",
    zlib.compress(bytes(rows))) + chunk(b"IEND", b"")

def main():
    public = ROOT / "public"
    public.mkdir(exist_ok=True)
    for size in (192, 512):
        path = public / f"pwa-icon-{size}.png"
        path.write_bytes(icon(size))
        print(f"Created {path.relative_to(ROOT)}")


if __name__ == "__main__":
    main()
```

Say: "Create the icon generator. It writes two PNG files using the standard library. Run it before the development server; production builds invoke it automatically."

Type: Follow the action above in `scripts/create_pwa_icons.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 2: CREATE - make parent folders, then create this file - ui/components/PwaStatus.html

Create the optional PWA status component with scoped CSS. Explain online/offline events, the waiting worker and cleanup before wiring the update button.

Create the parent directories, then save this complete file.

```
<template>
  <aside v-if="offline || updateAvailable || message" class="pwa-status"
    aria-label="Application status">
    <p v-if="offline" role="status">You are offline. Reconnect before saving changes.</p>
    <p v-if="updateAvailable" role="status">A new version is ready.</p>
    <button v-if="updateAvailable" type="button" :disabled="!applyUpdate"
    @click="applyUpdate()">Update and reload</button>
    <p v-if="message" role="status">{{ message }}</p>
  </aside>
</template>
<script lang="ts">
export default {
  data() {
    return { offline: !navigator.onLine, updateAvailable: false, message: "", cleanup: null,
    applyUpdate: null };
  },
  mounted() {
    let active = true;
    let reloadListener = null;
    const networkChanged = () => { this.offline = !navigator.onLine; };
    const updateReady = () => { this.updateAvailable = true; };
    const registrationFailed = () => { this.message = "Offline support could not start. You
    can still use the online app."; };
    window.addEventListener("online", networkChanged);
    window.addEventListener("offline", networkChanged);
```

```
    document.addEventListener("teloce:pwa-update", updateReady);
    document.addEventListener("teloce:pwa-error", registrationFailed);

    Promise.resolve(globalThis.telocePlugins?.pwa).then(controller => {
      if (!active || !controller) return;
      this.updateAvailable = controller.updateAvailable;
      this.applyUpdate = () => {
        if (!window.confirm("Save unfinished work first. Reload the application now?"))
    return;

        reloadListener = () => window.location.reload();
        navigator.serviceWorker.addEventListener("controllerchange", reloadListener, { once:
    true });
        controller.activateUpdate();
      };
    });
    this.cleanup = () => {
      active = false;
      window.removeEventListener("online", networkChanged);
      window.removeEventListener("offline", networkChanged);
      document.removeEventListener("teloce:pwa-update", updateReady);
      document.removeEventListener("teloce:pwa-error", registrationFailed);
      if (reloadListener) navigator.serviceWorker.removeEventListener("controllerchange",
    reloadListener);
    };
  },
  unmounted() {
    this.cleanup?.();
  },
};
</script>
<style scoped>
.pwa-status { padding: 1rem; background: #dbeafe; color: #17243a; border-bottom: 1px solid
    #93c5fd; }
p { margin: 0.25rem 0; }
button { margin-top: 0.5rem; padding: 0.6rem 1rem; border: 0; border-radius: 0.4rem;
    background: #1464aa; color: white; cursor: pointer; }
button:focus-visible { outline: 3px solid #f59e0b; outline-offset: 3px; }
</style>
```

Say: "Create the optional PWA status component with scoped CSS. Explain online/offline events, the waiting worker and cleanup before wiring the update button."

Type: Follow the action above in `ui/components/PwaStatus.html`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 3: EDIT - replace the marked blocks - app.py

The application factory mounts only the features already introduced. Follow the action label: a marked-block edit keeps the surrounding factory intact.

Edit 1: find this exact block

Open `app.py`. Locate this block once. The unchanged surrounding lines identify its position; do not use line numbers from another revision.

```
        admin_path or ADMIN_DATABASE_PATH,
        Path(admin_path or ADMIN_DATABASE_PATH).parent / "uploads",
    )
```

```
app.use_teloce(
    project_root=ROOT,
    ui_dir="ui",
    title="Project Manager",
    options={"minifier": "minifyjs"},
)

@app.get("/health/course")
```

Replace that block with

```
    admin_path or ADMIN_DATABASE_PATH,
    Path(admin_path or ADMIN_DATABASE_PATH).parent / "uploads",
)
app.mount_static("/course-assets", str(ROOT / "public"), cache_control="no-cache")
app.use_teloce(
    project_root=ROOT,
    ui_dir="ui",
    title="Project Manager",
    options={"minifier": "minifyjs"},
    plugins={
        "pwa": {
            "name": "Flaxon Project Manager",
            "short_name": "Projects",
            "start_url": "/projects",
            "scope": "/",
            "theme_color": "#17243a",
            "icons": [
                {"src": "/course-assets/pwa-icon-192.png", "sizes": "192x192", "type":
"image/png", "purpose": "any maskable"},
                {"src": "/course-assets/pwa-icon-512.png", "sizes": "512x512", "type":
"image/png", "purpose": "any maskable"},
            ],
        },
    },
)

@app.get("/health/course")
```

Leave the rest of the file unchanged.

Say: "The application factory mounts only the features already introduced. Follow the action label: a marked-block edit keeps the surrounding factory intact."

Type: Follow the action above in `app.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 4: EDIT - replace the marked blocks - ui/app.html

This is the SPA shell. Its internal anchors and router view cooperate with each module's page mappings.

Edit 1: find this exact block

Open `ui/app.html`. Locate this block once. The unchanged surrounding lines identify its position; do not use line numbers from another revision.

```
      <a href="/login" data-teloce-link>Account</a>
    </nav>
  </header>
```

```
    <main id="router-view" data-teloce-router-view></main>
    <footer>Built with Flaxon and Teloce. <a href="/admin/">Staff Admin</a></footer>
  </div>
</template>
<script lang="ts">
export default {};
</script>
<style scoped>
/* Only the document reset is global: body is outside this component. */
```

Replace that block with

```
        <a href="/login" data-teloce-link>Account</a>
      </nav>
    </header>
    <PwaStatus />
    <main id="router-view" data-teloce-router-view></main>
    <footer>Built with Flaxon and Teloce. <a href="/admin/">Staff Admin</a></footer>
  </div>
</template>
<script lang="ts">
import PwaStatus from "./components/PwaStatus.html";
export default { components: { PwaStatus } };
</script>
<style scoped>
/* Only the document reset is global: body is outside this component. */
```

Leave the rest of the file unchanged.

Say: "This is the SPA shell. Its internal anchors and router view cooperate with each module's page mappings."

Type: Follow the action above in `ui/app.html`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 5: EDIT - replace the marked blocks - scripts/build_ui.py

This file owns the feature named by its module or component. Read the route/form flow before continuing.

Edit 1: find this exact block

Open `scripts/build_ui.py`. Locate this block once. The unchanged surrounding lines identify its position; do not use line numbers from another revision.

```
    os.environ["DATA_DIR"] = build_data
    os.environ["FLAXON_DEBUG"] = "0"
    os.environ.setdefault("FLAXON_SECRET_KEY", secrets.token_urlsafe(48))
    from app import app

    result = app.teloce.build()
```

Replace that block with

```
    os.environ["DATA_DIR"] = build_data
    os.environ["FLAXON_DEBUG"] = "0"
    os.environ.setdefault("FLAXON_SECRET_KEY", secrets.token_urlsafe(48))
    from create_pwa_icons import main as create_icons

    create_icons()
    from app import app

    result = app.teloce.build()
```

Leave the rest of the file unchanged.

Say: "This file owns the feature named by its module or component. Read the route/form flow before continuing."

Type: Follow the action above in `scripts/build_ui.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 6: EDIT - replace the marked blocks - scripts/browser_smoke.py

This file owns the feature named by its module or component. Read the route/form flow before continuing.

Edit 1: find this exact block

Open `scripts/browser_smoke.py`. Locate this block once. The unchanged surrounding lines identify its position; do not use line numbers from another revision.

```
def main():
    with tempfile.TemporaryDirectory() as directory:
        production = "--production" in sys.argv
        env = {
```

Replace that block with

```
def main():
    from create_pwa_icons import main as create_icons
    create_icons()
    with tempfile.TemporaryDirectory() as directory:
        production = "--production" in sys.argv
        env = {
```

Leave the rest of the file unchanged.

Say: "This file owns the feature named by its module or component. Read the route/form flow before continuing."

Type: Follow the action above in `scripts/browser_smoke.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 7: CREATE - make parent folders, then create this file - tests/test_pwa.py

Verify the manifest, actual PNG responses, worker scope and development registration policy before recording installation.

Create the parent directories, then save this complete file.

```
"""Verify the course's optional PWA assets and production configuration."""
import json
import runpy
from pathlib import Path
from flaxon.testing import TestClient

ROOT = Path(__file__).resolve().parents[1]


def test_course_pwa_assets(application):
    runpy.run_path(str(ROOT / "scripts/create_pwa_icons.py"), run_name="__main__")
    integration = application.teloce
    result = integration.build()
    assert result["plugins"]["enabled"] == ["pwa"]
    client = TestClient(application)
    shell = client.get("/")
    assert shell.status_code == 200
    assert 'rel="manifest"' in shell.text
    manifest = json.loads(client.get("/_flaxon/teloce-plugins/manifest.webmanifest").text)
    assert manifest["start_url"] == "/projects"
    assert {icon["sizes"] for icon in manifest["icons"]} == {"192x192", "512x512"}
    for icon in manifest["icons"]:
        response = client.get(icon["src"])
        assert response.status_code == 200
        assert response.content.startswith(b"\x89PNG\r\n\x1a\n")
    worker = client.get("/teloce-sw.js")
    assert worker.headers["service-worker-allowed"] == "/"
    assert "application/javascript" in worker.headers["content-type"]
    assert "/api/" not in worker.text
    assert "/admin/" not in worker.text
    plugin = client.get("/_flaxon/teloce-plugins/index.js").text
    assert '"enabled": false' in plugin  # Debug builds do not register a worker.
```

Say: "Verify the manifest, actual PNG responses, worker scope and development registration policy before recording installation."

Type: Follow the action above in `tests/test_pwa.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Step 8: CREATE - make parent folders, then create this file - scripts/pwa_smoke.py

Create the disposable production browser check. It uses a temporary database, verifies cache exclusions, goes offline and reconnects.

Create the parent directories, then save this complete file.

```
"""Check production PWA registration and offline fallback using disposable data."""
import os
from pathlib import Path
```

```
import secrets
import subprocess
import sys
import tempfile
import time

import httpx
from playwright.sync_api import sync_playwright
from create_pwa_icons import main as create_icons

ROOT = Path(__file__).resolve().parents[1]
BASE = "http://127.0.0.1:8124"


def main():
    create_icons()
    with tempfile.TemporaryDirectory(prefix="course-pwa-") as directory:
        environment = {
            **os.environ,
            "DATA_DIR": directory,
            "FLAXON_DEBUG": "0",
            "FLAXON_SECRET_KEY": secrets.token_urlsafe(48),
            "PUBLIC_ORIGIN": BASE,
        }
        subprocess.run([sys.executable, "management.py", "migrate"], cwd=ROOT,
    env=environment, check=True)
        with open(Path(directory) / "server.log", "w") as log:
            server = subprocess.Popen([sys.executable, "-m", "uvicorn", "app:app", "--port",
    "8124"], cwd=ROOT, env=environment, stdout=log, stderr=log)
            try:
                for _ in range(150):

                    try:
                        if httpx.get(BASE + "/login", trust_env=False).status_code == 200:
                            break
                    except httpx.ConnectError:
                        pass
                    time.sleep(0.1)
                else:
                    raise RuntimeError((Path(directory) / "server.log").read_text())
                with sync_playwright() as playwright:
                    browser = playwright.chromium.launch()
                    context = browser.new_context(service_workers="allow")
                    page = context.new_page()
                    page.goto(BASE + "/login")
                    page.evaluate("async () => { await
    Promise.race([navigator.serviceWorker.ready, new Promise((_, reject) => setTimeout(() =>
    reject(new Error('Service worker did not become ready')), 15000))]); }")
                    page.reload()
                    page.wait_for_function("navigator.serviceWorker.controller !== null")
                    assert page.locator('link[rel="manifest"]').count() == 1
                    cached = page.evaluate("async () => { const names = await caches.keys();
    const urls = []; for (const name of names) { for (const request of await (await
    caches.open(name)).keys()) urls.push(new URL(request.url).pathname); } return urls; }")
                    assert cached and any(path.endswith("offline.html") for path in cached)
                    assert not any(path.startswith(("/api/", "/admin")) for path in cached)
                    # Browser offline emulation can leave worker fetches online.
                    # Stop the server so this checks an actual network failure.
                    server.terminate()
                    server.wait(timeout=10)
                    context.set_offline(True)
                    page.goto(BASE + "/projects")
                    assert page.get_by_role("heading", name="You are offline").is_visible()
                    context.set_offline(False)
                    server = subprocess.Popen([sys.executable, "-m", "uvicorn", "app:app",
```

```
    "--port", "8124"], cwd=ROOT, env=environment, stdout=log, stderr=log)

                for _ in range(150):
                    try:
                        if httpx.get(BASE + "/login", trust_env=False).status_code == 200:
                            break
                    except httpx.ConnectError:
                        pass
                    time.sleep(0.1)
                else:
                    raise RuntimeError("Server did not restart after the offline check")
                page.goto(BASE + "/login")
                assert page.get_by_label("Email", exact=True).is_visible()
                context.close()
                browser.close()
                print("PWA registration, private-cache exclusion, offline fallback and
    reconnect passed.")
            finally:
                server.terminate()
                try:
                    server.wait(timeout=10)
                except subprocess.TimeoutExpired:
                    server.kill()
                    server.wait()


if __name__ == "__main__":
    main()
```

Say: "Create the disposable production browser check. It uses a temporary database, verifies cache exclusions, goes offline and reconnects."

Type: Follow the action above in `scripts/pwa_smoke.py`; explain each new function or binding as you add it.

Viewers should see: The saved file matches this step. After all steps, run the chapter checks below and show the expected result.

Run and check

```
python scripts/create_pwa_icons.py
python scripts/build_ui.py
python -m pytest -q tests/test_pwa.py
python scripts/pwa_smoke.py
```

Expected result

The manifest has two valid icons and starts at /projects. The production browser registers /teloce-sw.js. An offline navigation displays You are offline; reconnect restores the login page. A waiting update shows a button, and reload happens only after the user confirms.

Common errors

Use the refreshed preview wheels: older Teloce builds do not implement this plugin. HTTPS or localhost is required. Generate icons before starting the app. Debug mode intentionally registers no worker. Browser installation eligibility varies. The service worker does not provide offline writes. Check DevTools Application and use Unregister/Clear storage when switching between production and development; do not clear real user data without their consent.

Short exercise

Deploy a second version to a disposable HTTPS site. Leave the first version open, request an update in DevTools, and show that the confirmation protects unfinished work.

What to say and show

Say: "Turn the completed SPA into an optional installable application without caching private data. The server remains responsible for persistence and authorization; the browser presents the result."

Show the expected result above, then point to the file responsible for it. Run the chapter check before the next lesson.

Save your checkpoint

```
git add .
git commit -m "Complete chapter 16: PWA: installation, offline fallback and updates"
```

Appendix | Recording this application

Rehearse the complete build once before recording. Keep this ebook beside the editor. For each lesson: demonstrate the current result, introduce the file, explain one function or component at a time, type its behavior, run the check and show the working result. Provide full scoped CSS for copying, then explain a few relevant rules while recording.

Record a short sample using chapter 5's owned project endpoint, its HTTP demo, and chapter 9's ProjectList form. Tell viewers which earlier chapters supply sessions and validation. Gather feedback on pacing and error explanations before recording all chapters.

The repository contains the completed application, generated learner starter, chapter files and verification script. `course/checkpoints/chapter-NN.json` supplies the exact complete files for chapters 2-16. Run `python scripts/restore_chapter.py 5 ../chapter-05-recovery` from course_reference to recover chapter 5 into a new folder. Existing work is never overwritten. Use the revision-6 chapter snapshots rather than the earlier SQL-course tags. A snapshot is for recovery, not a replacement for teaching the file changes.